

Software Engineering

How To Create Software Products Everyone Loves!

Module 17

Software Architecture and Technical Design: Architecture Styles

Ali Samanipour

May. 2023

Ali Samanipour
[linkedin.com/in/Samanipour](https://www.linkedin.com/in/Samanipour)

1

Introduction to Software Architecture

2

Defining Software Architecture & Architecture Characteristics

3

Layered Architecture Style

4

Microkernel Architecture Style

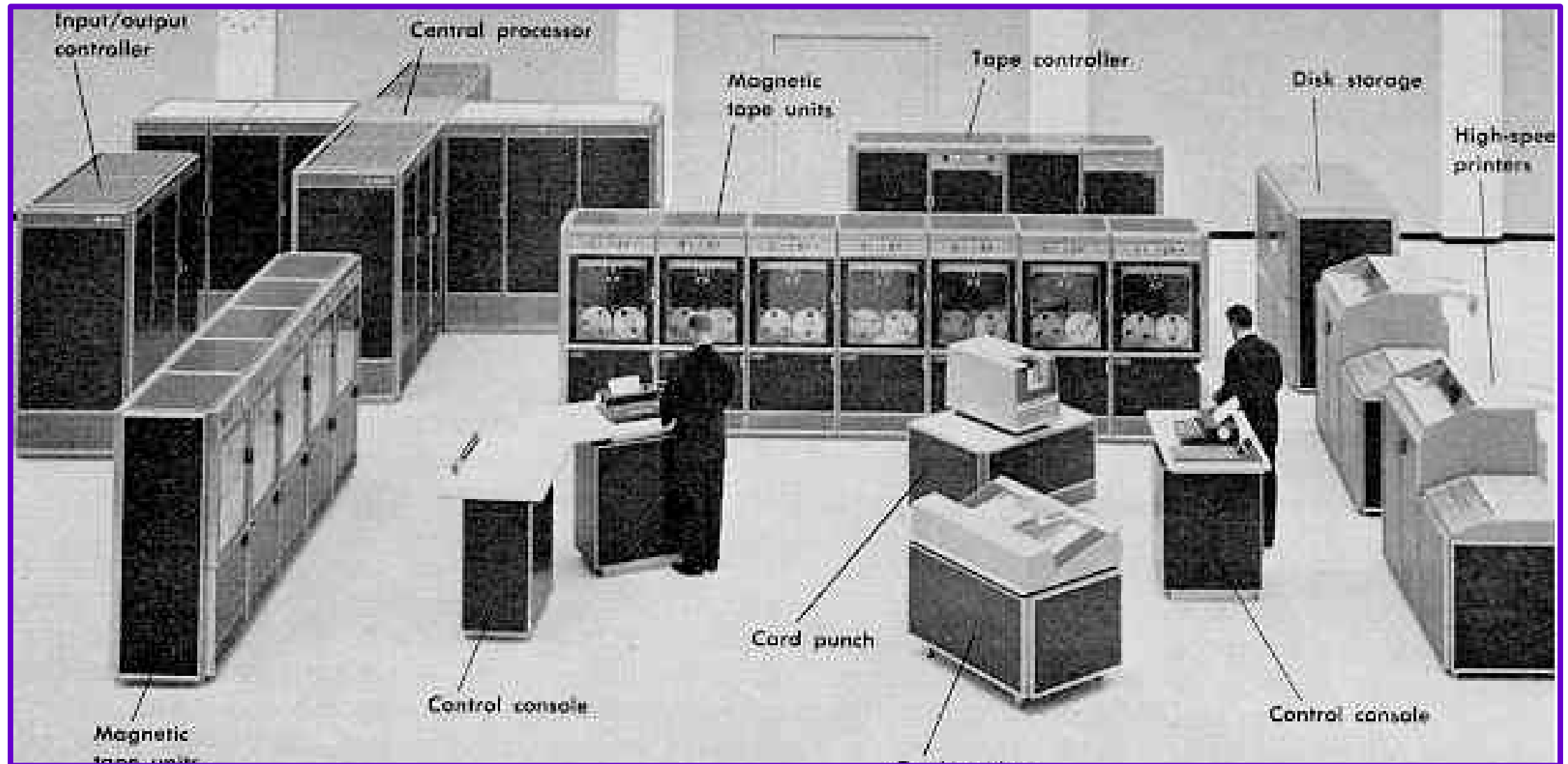
5

Service-Based Architecture Style

6

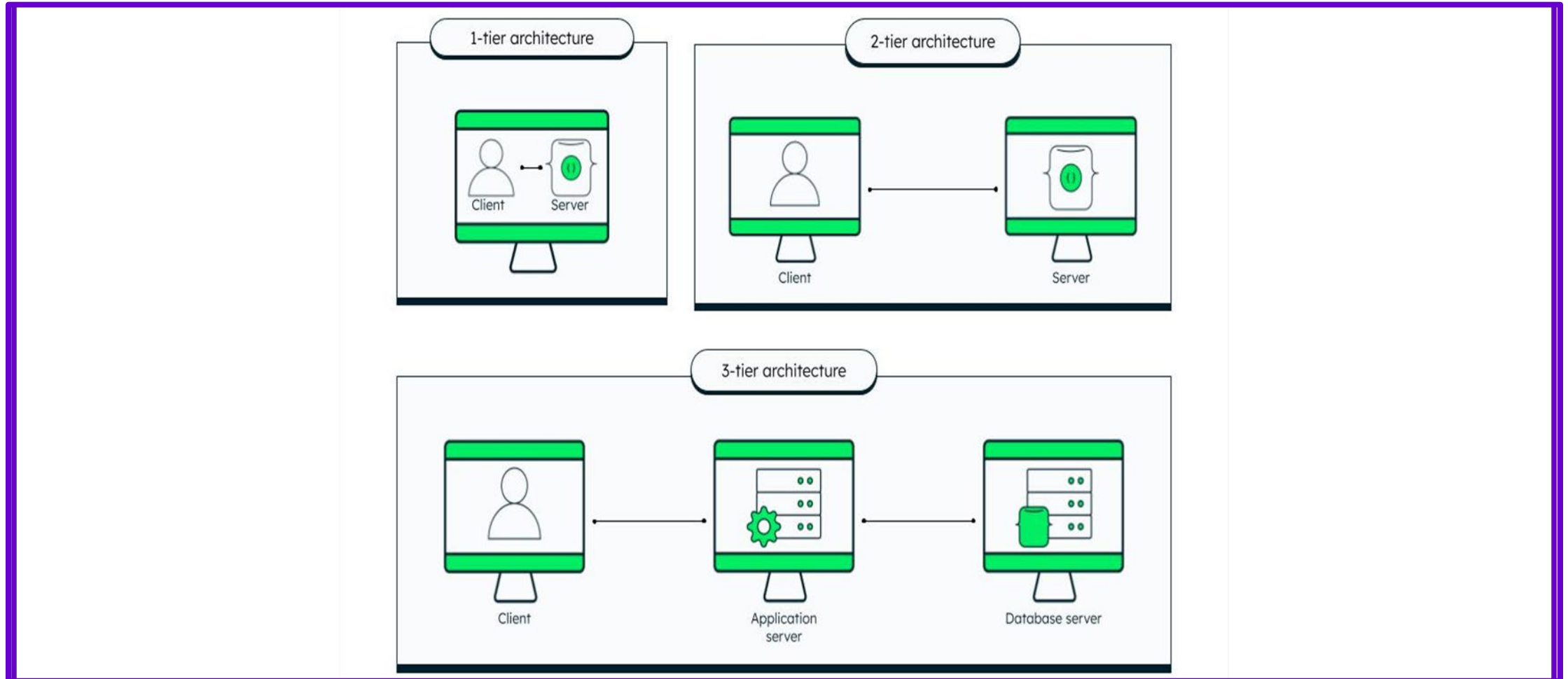
Event-Driven Architecture Style

Unitary Architecture

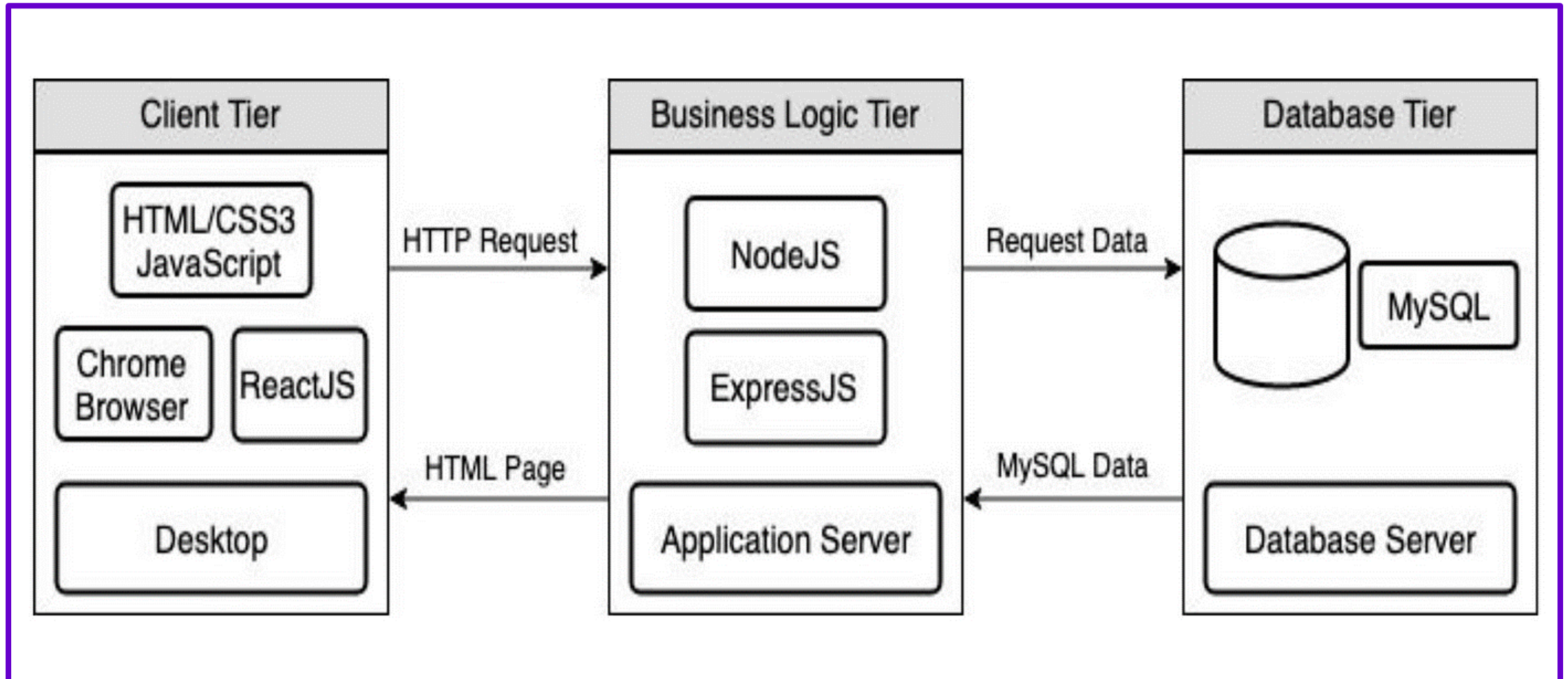


Client/Server Architecture

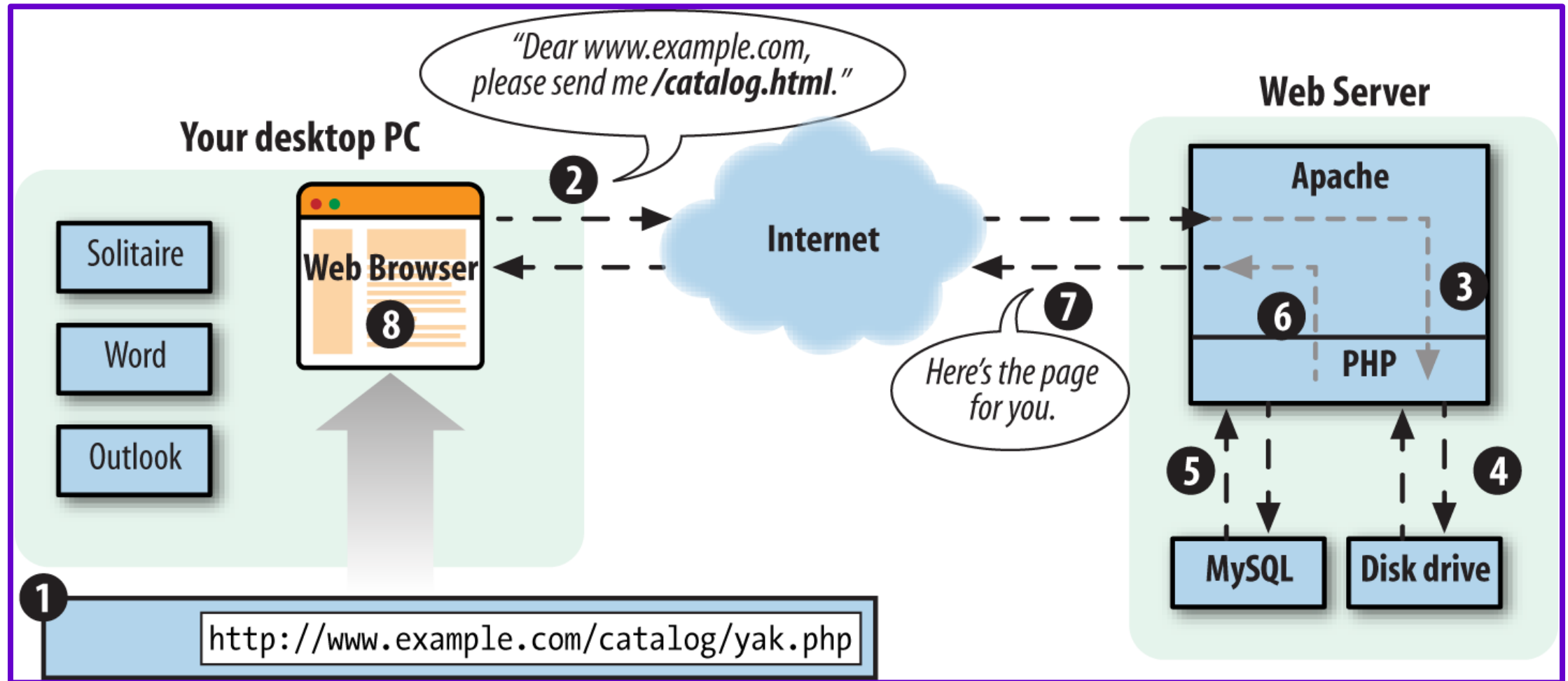
(Desktop + database server, Browser + web server, Three-tier)



Client/Server Architecture (Three-tier)

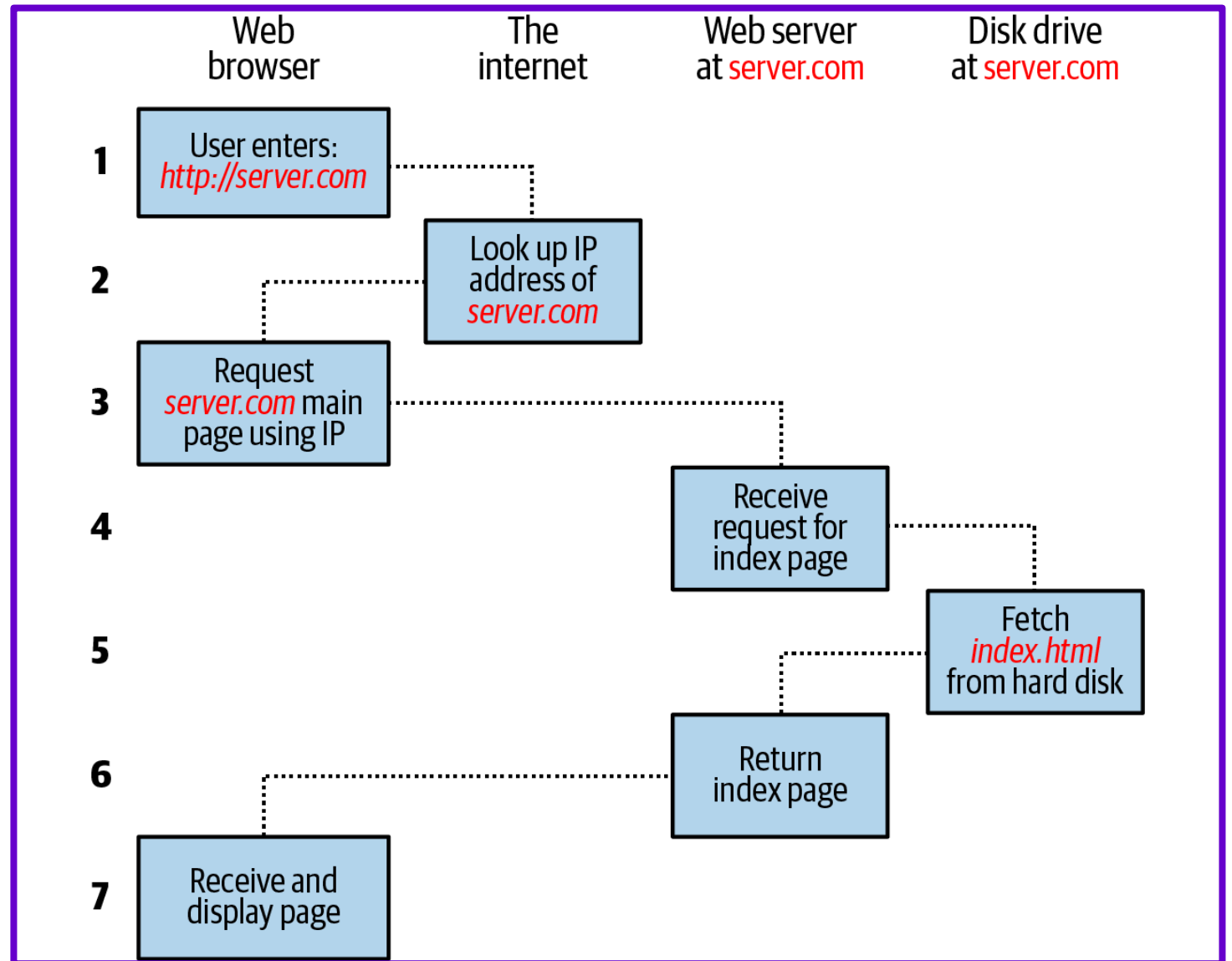


How Client/Server Architecture Works? (Web Apps)



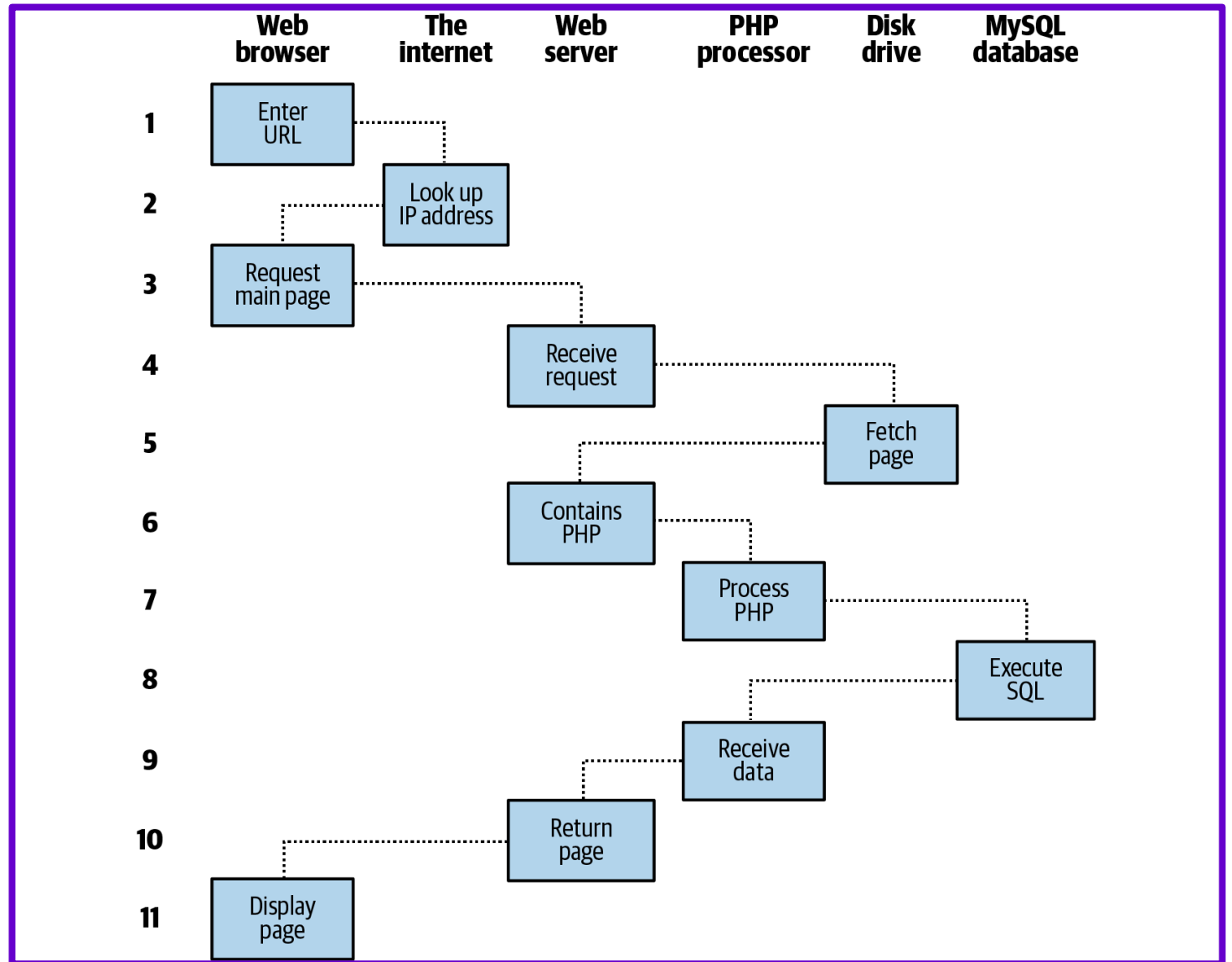
The Request/Response Procedure

At its most basic level, the request/response process consists of a web browser or other client asking the web server to send it a web page and the server sending back the page.



A dynamic client/server
request/response
sequence

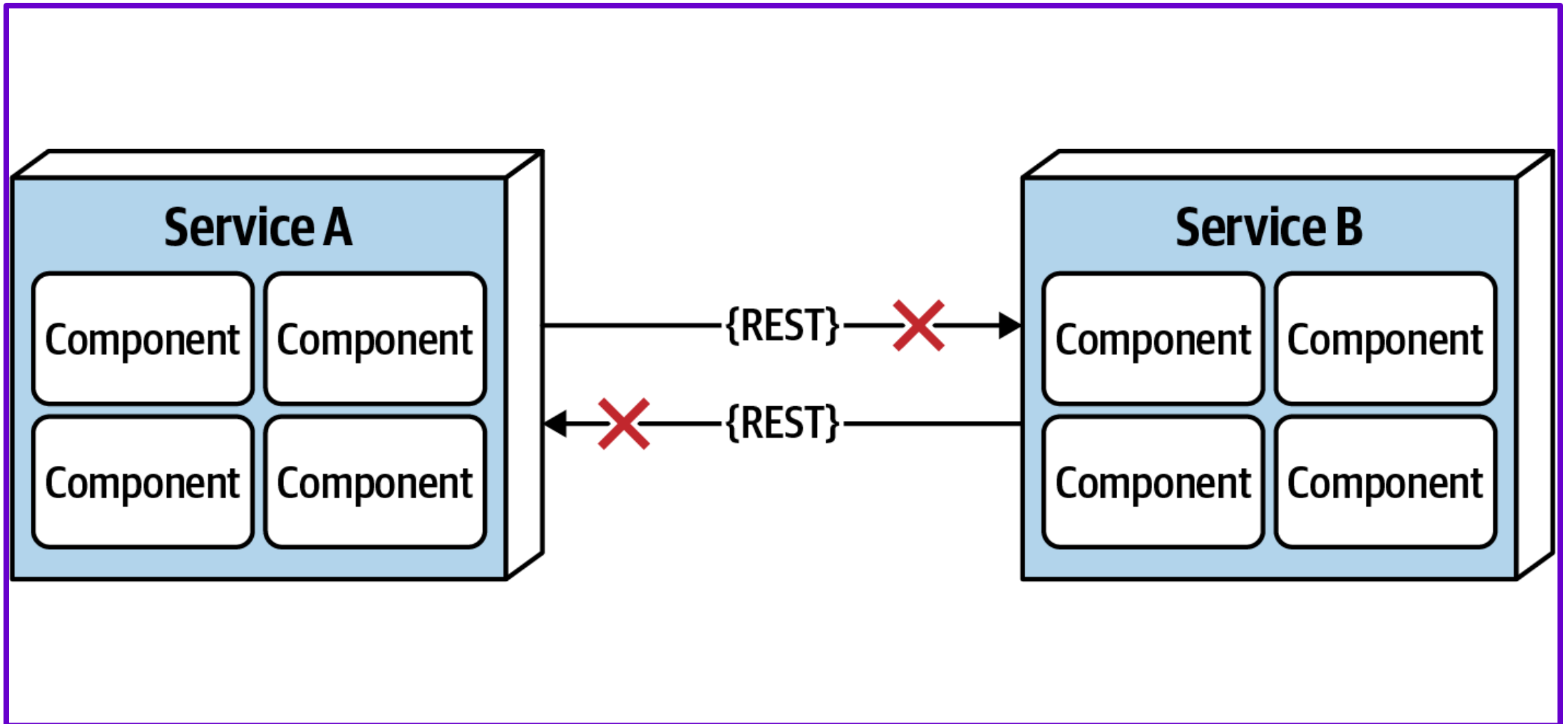
For dynamic web
pages, the
procedure is a little
more involved,
because it may
bring both PHP and
MySQL into the
mix.



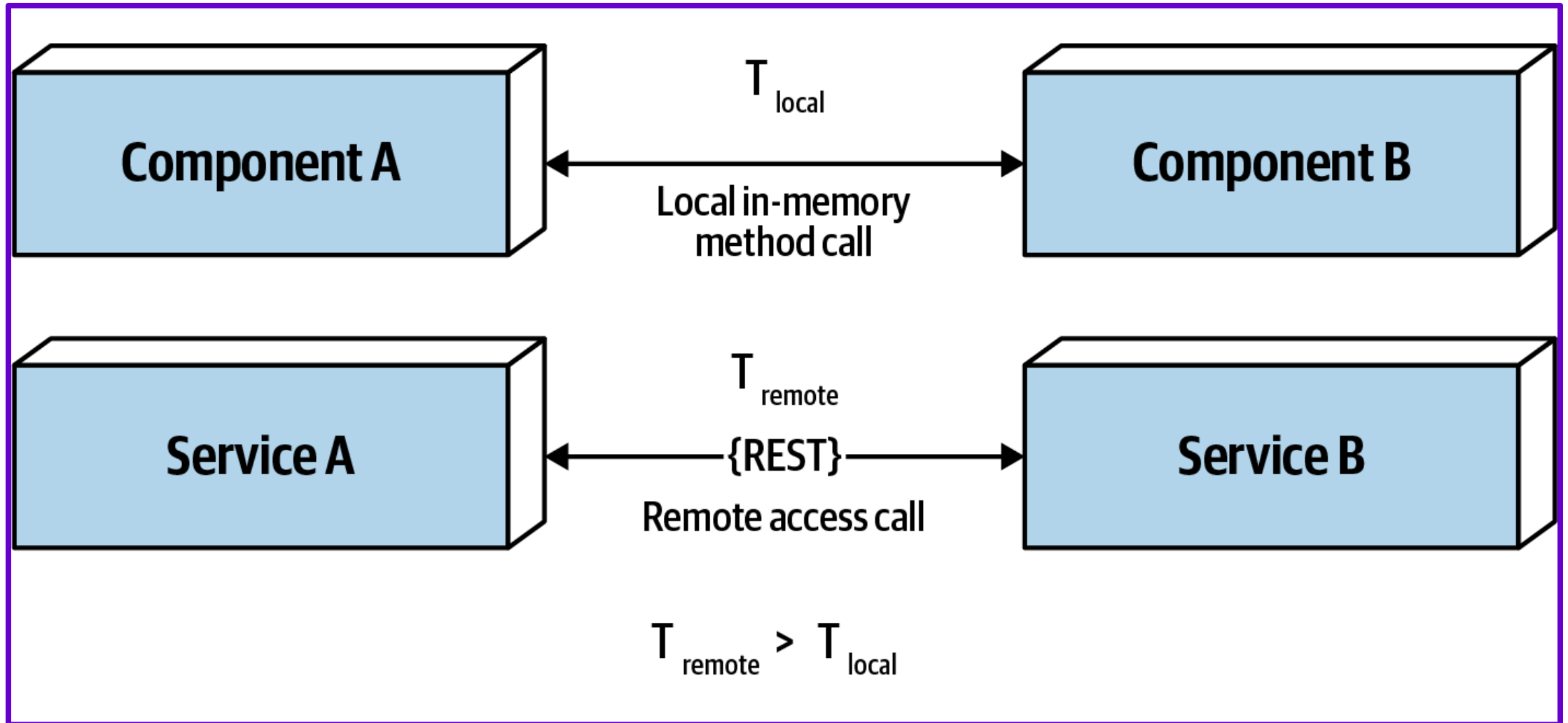
Monolithic Versus Distributed Architectures

Architecture styles can be classified into two main types: *monolithic* (single deployment unit of all code) and *distributed* (multiple deployment units connected through remote access protocols).

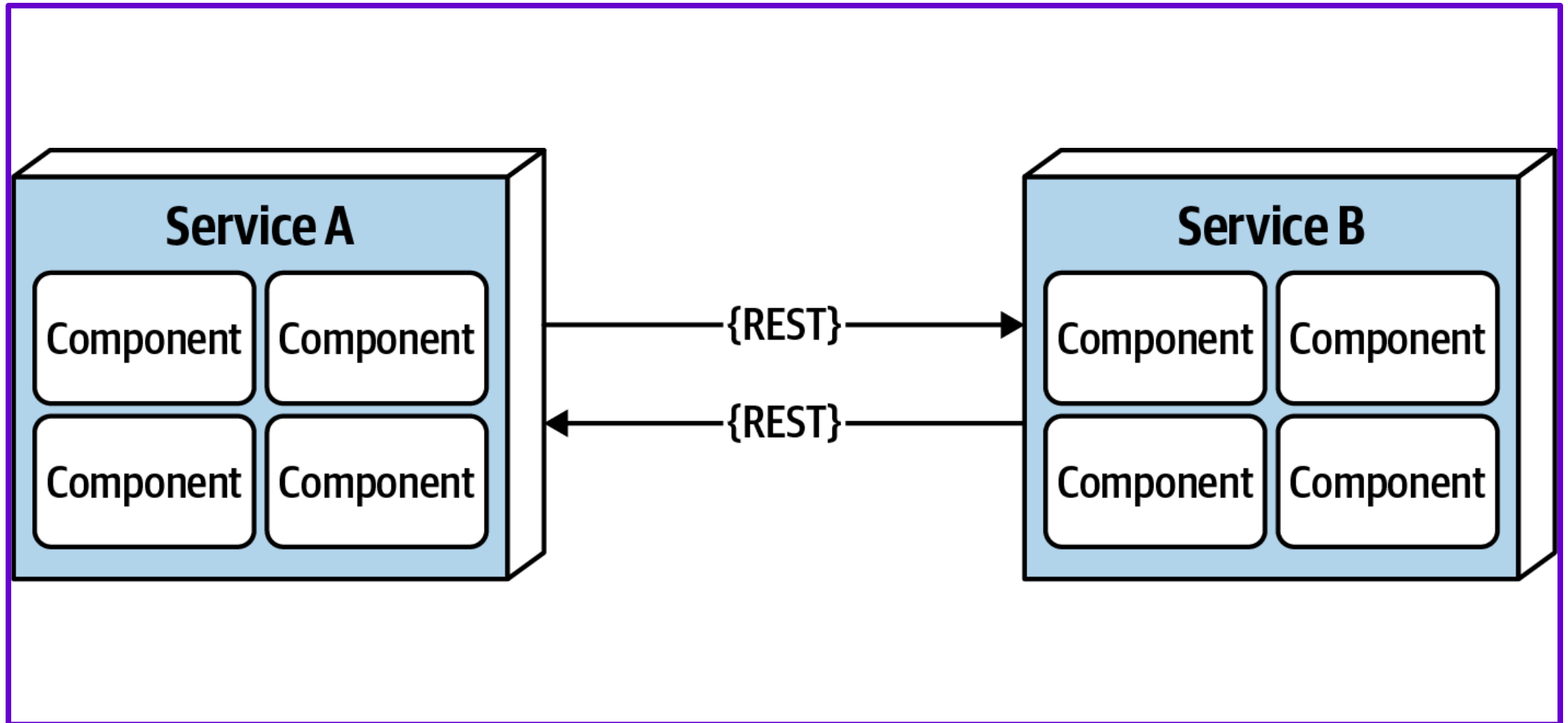
Fallacy #1: The Network Is Reliable



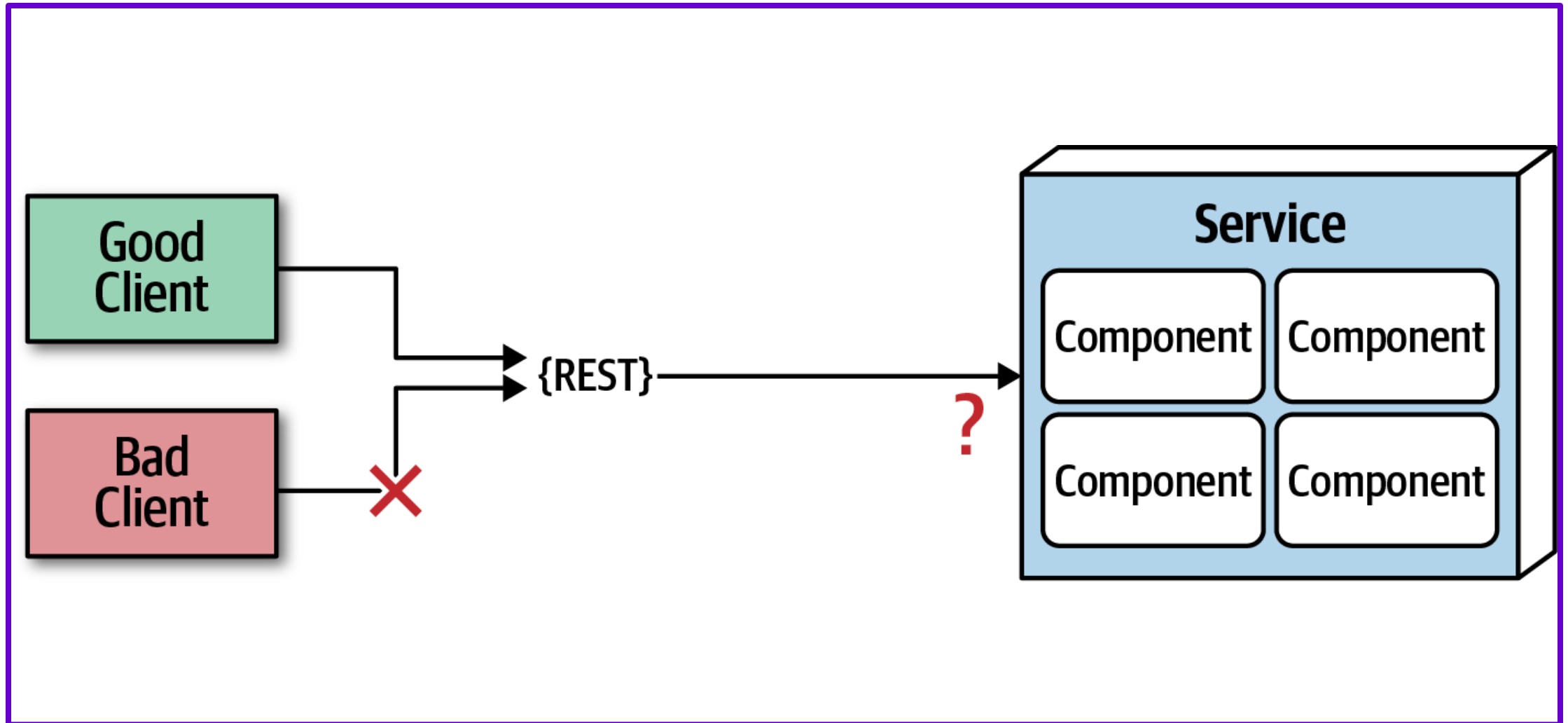
Fallacy #2: Latency Is Zero



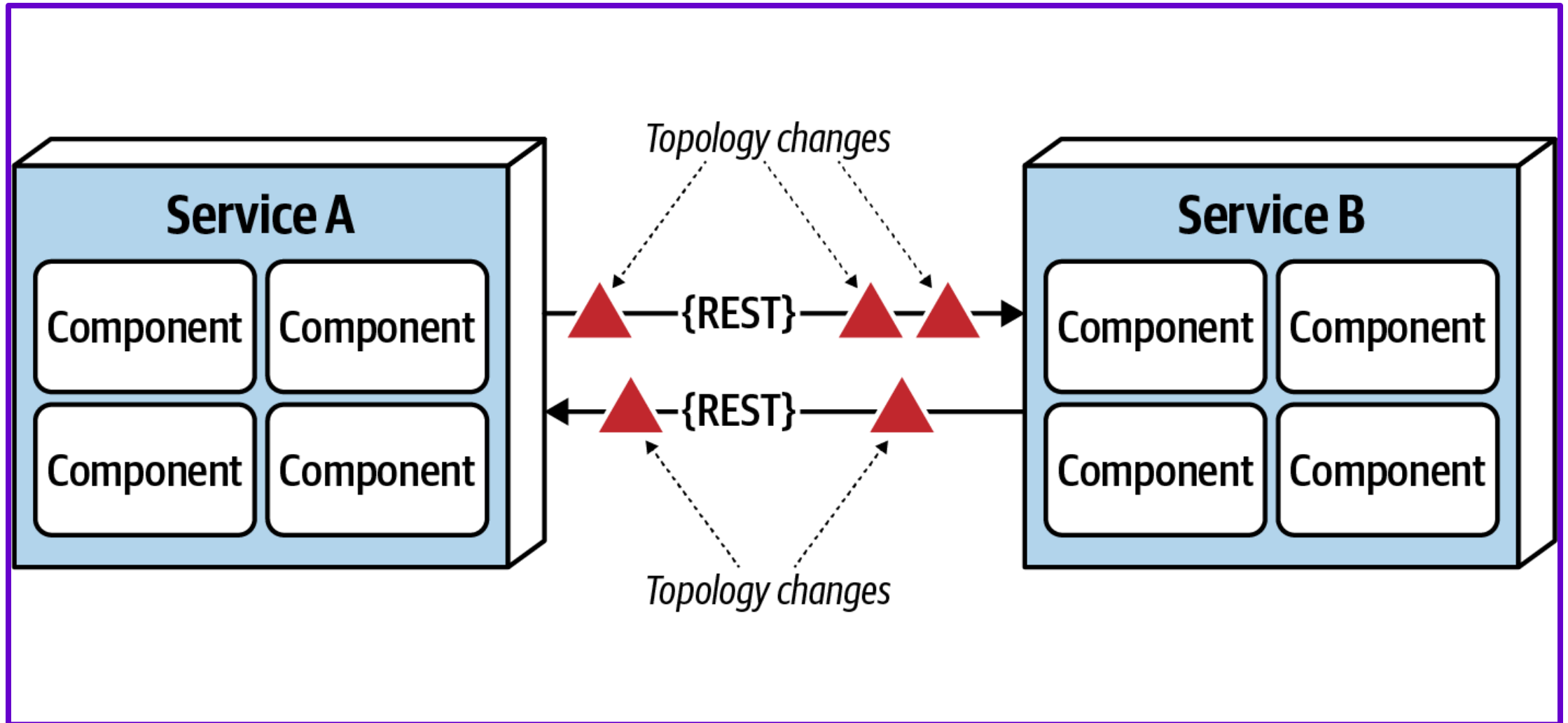
Fallacy #3: Bandwidth Is Infinite



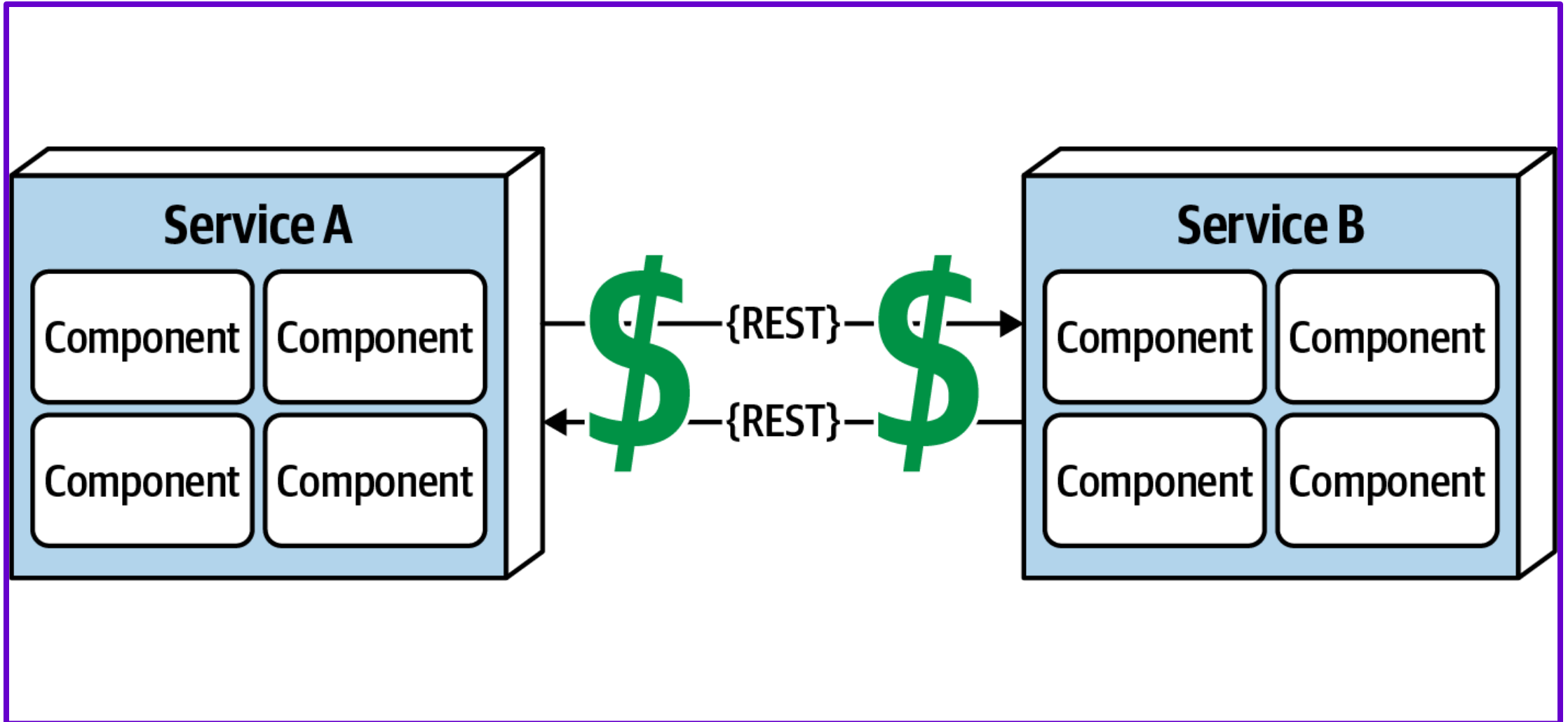
Fallacy #4: The Network Is Secure



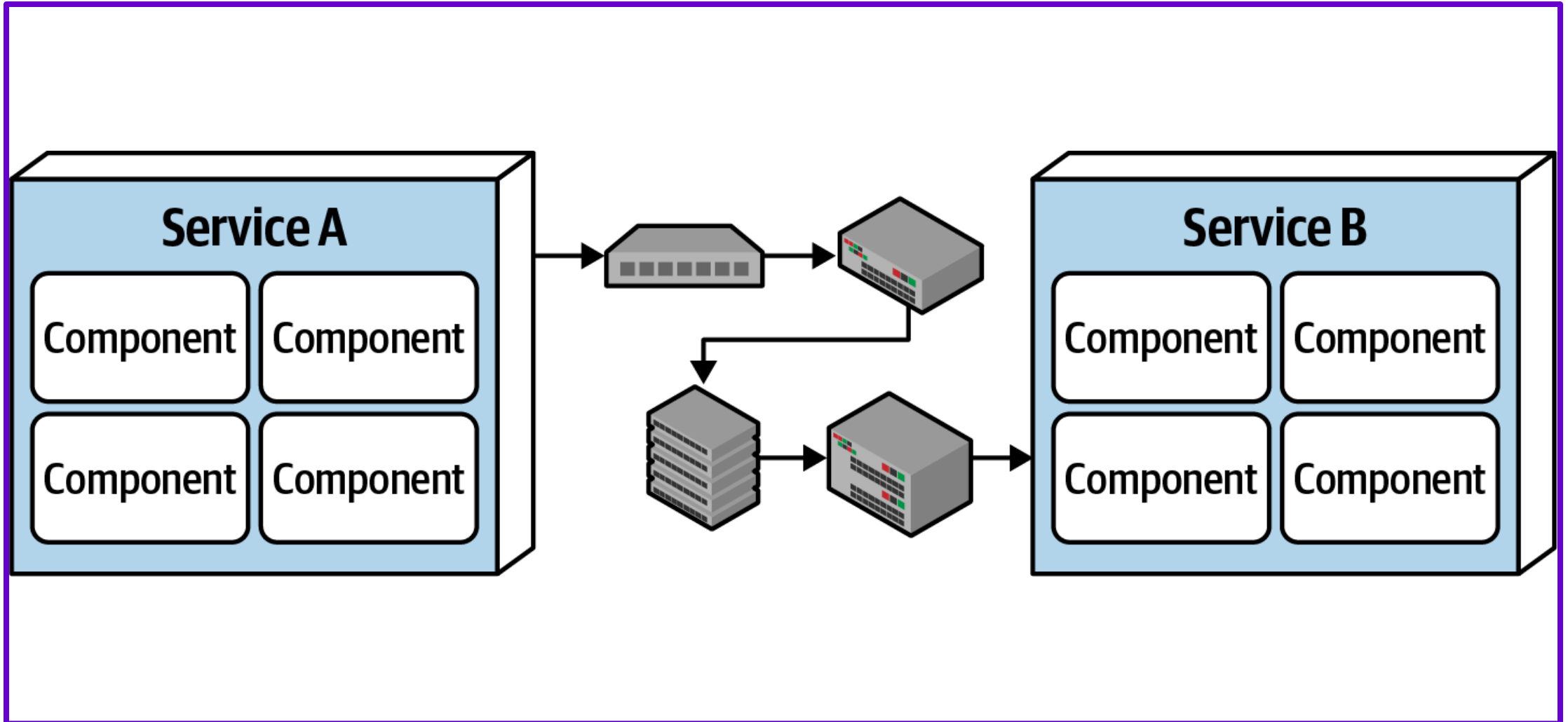
Fallacy #5: The Topology Never Changes



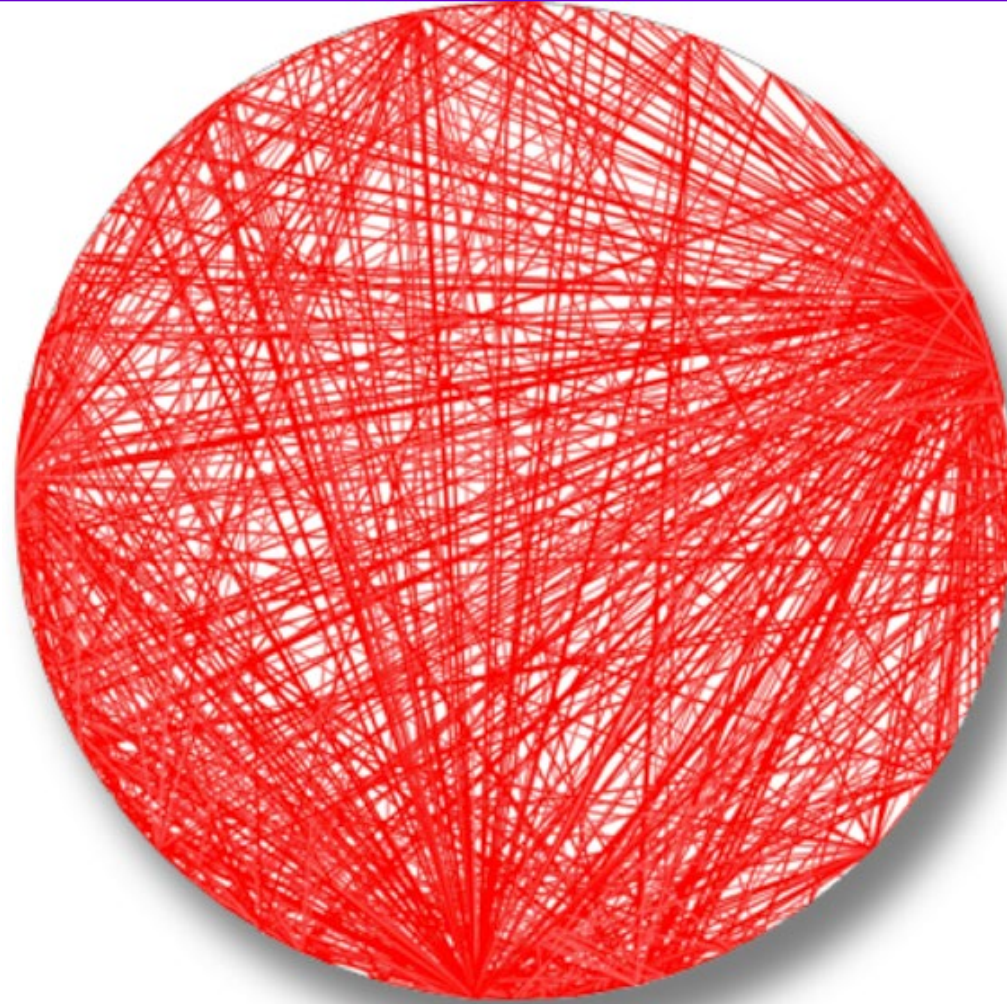
Fallacy #6: Transport Cost Is Zero



Fallacy #7: The Network Is Homogeneous



A Big Ball of Mud architecture visualized from a real code base



1

Introduction to Software Architecture

2

Defining Software Architecture & Architecture Characteristics

3

Layered Architecture Style

4

Microkernel Architecture Style

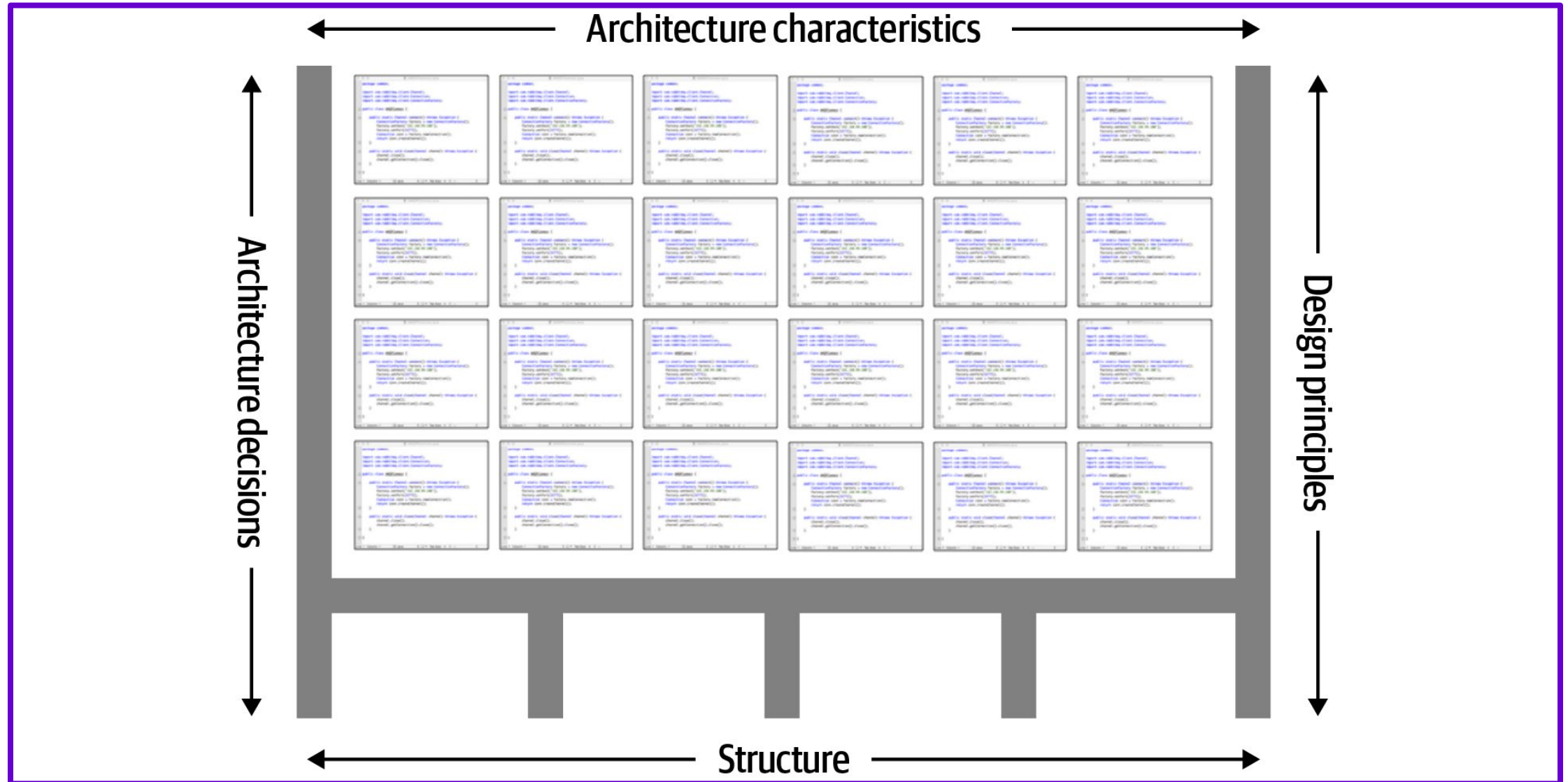
5

Service-Based Architecture Style

6

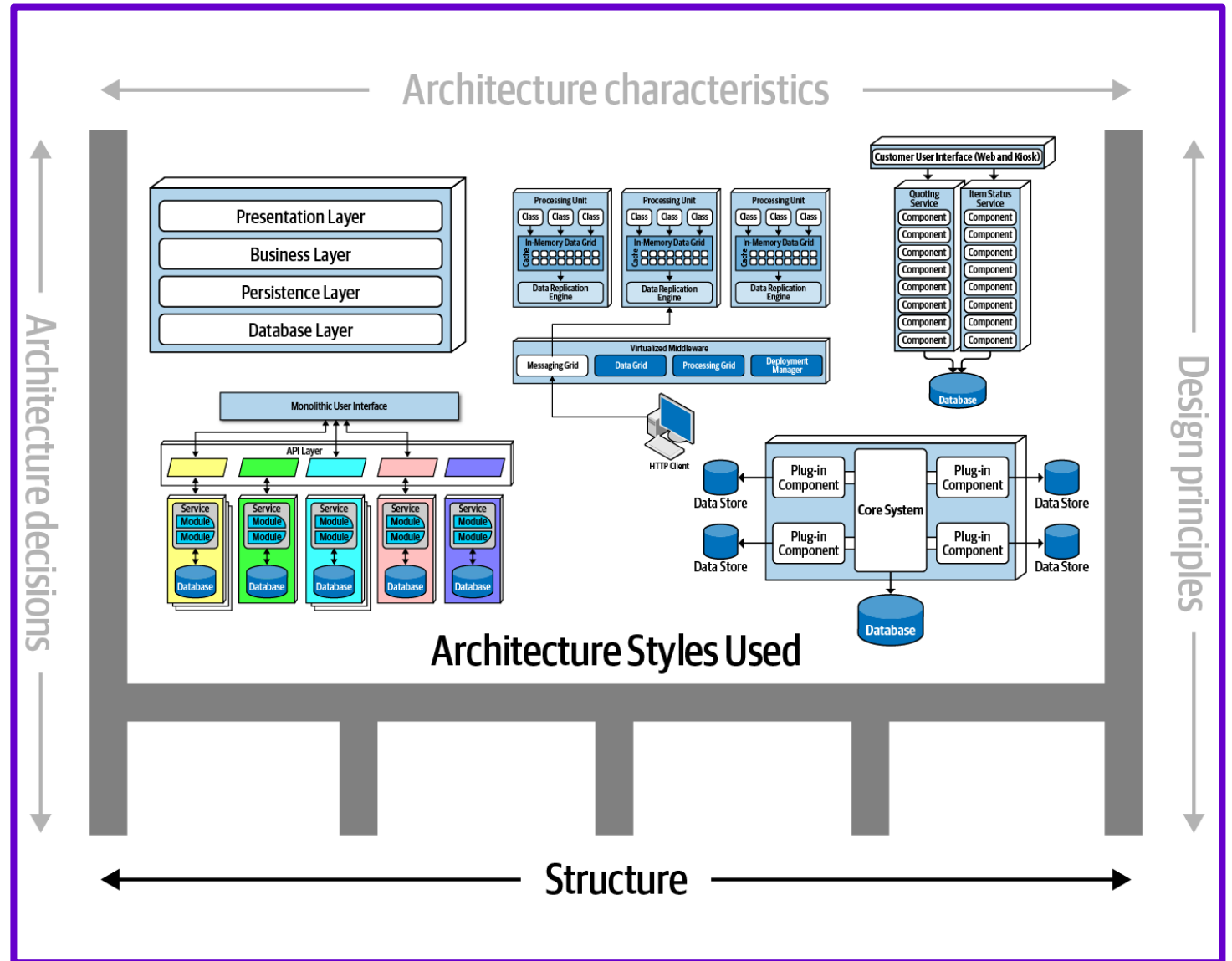
Event-Driven Architecture Style

Defining Software Architecture



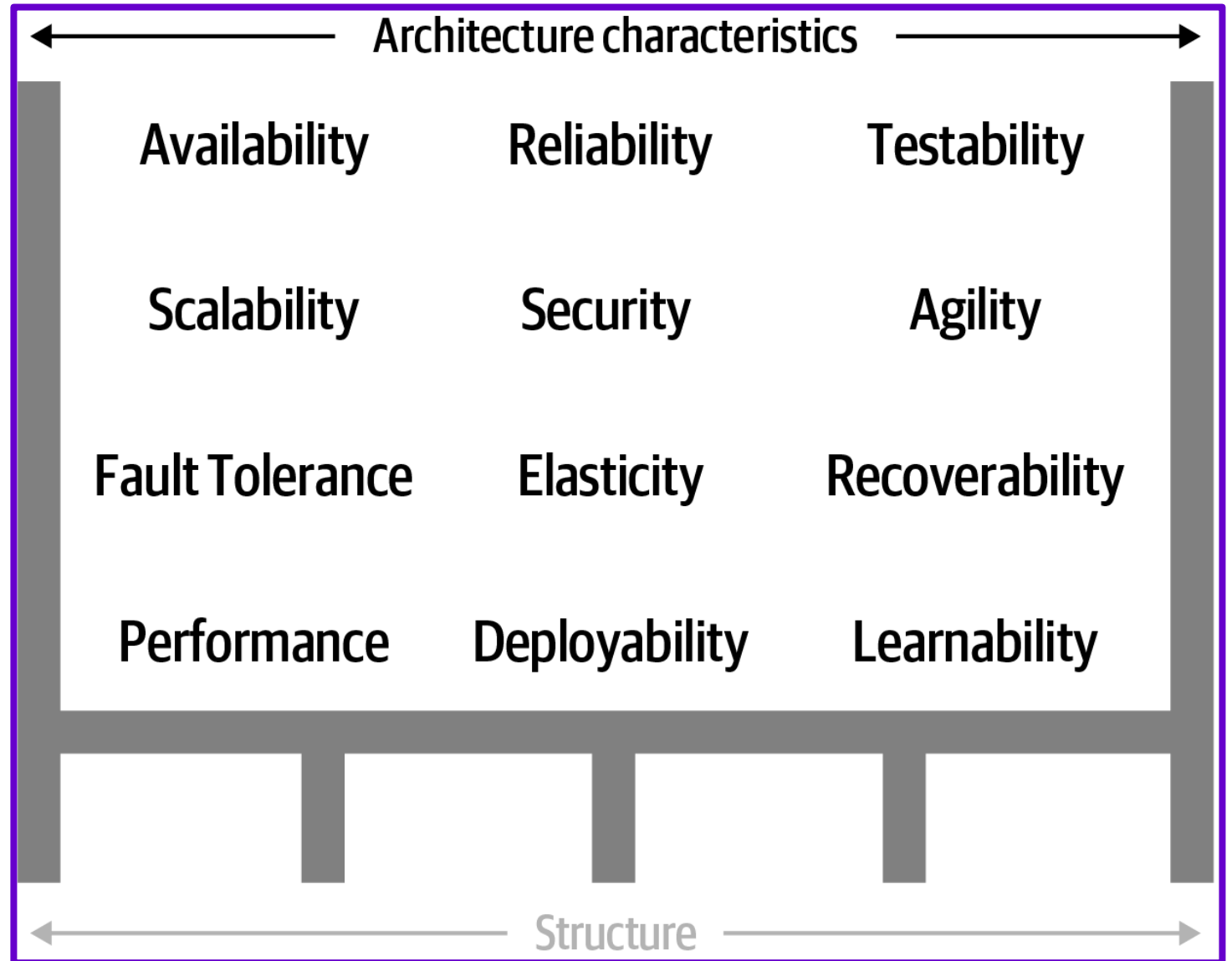
Defining Software Architecture

Structure refers to the type of architecture styles used in the system



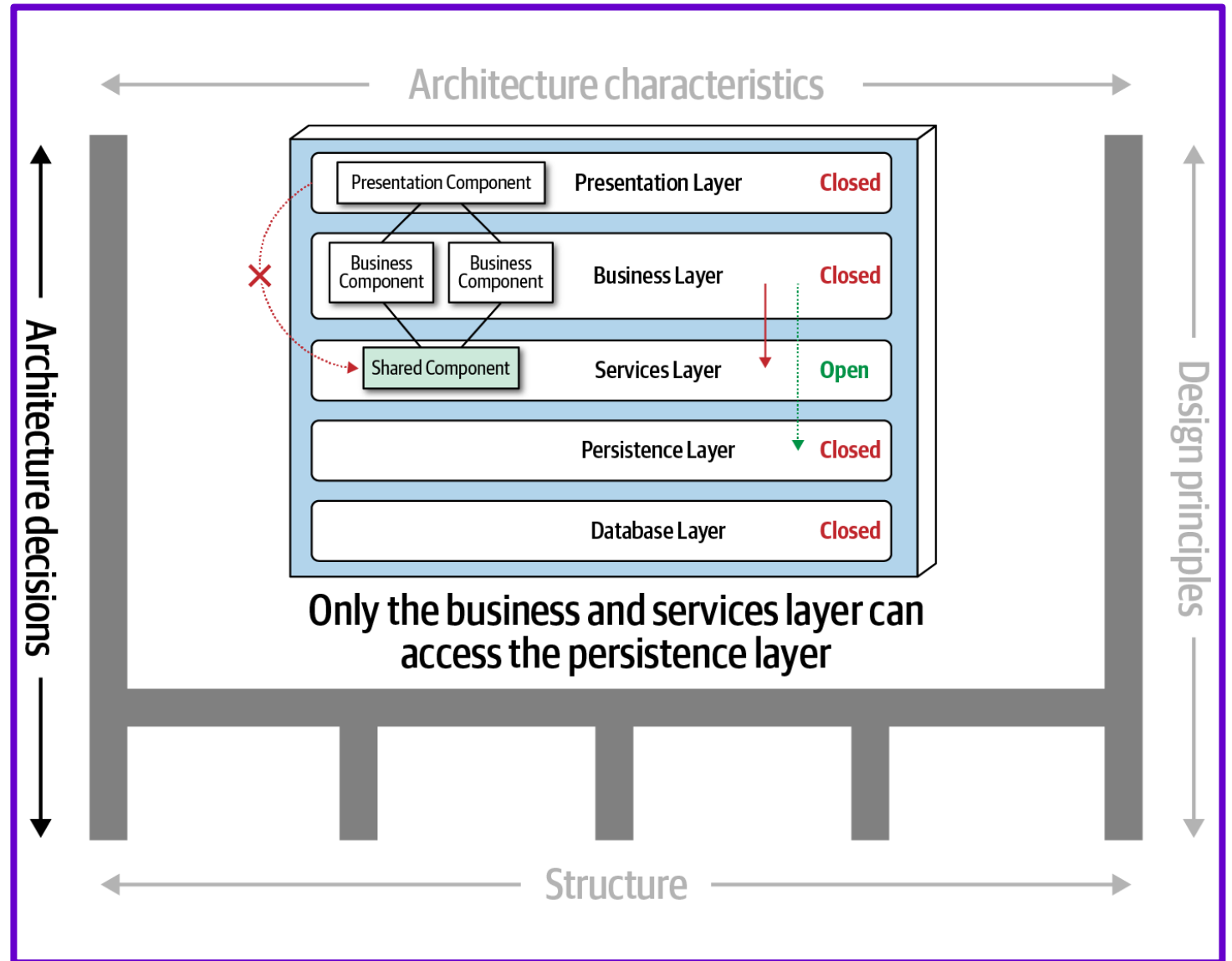
Defining Software Architecture

Architecture characteristics refers to the “-ilities” that the system must support



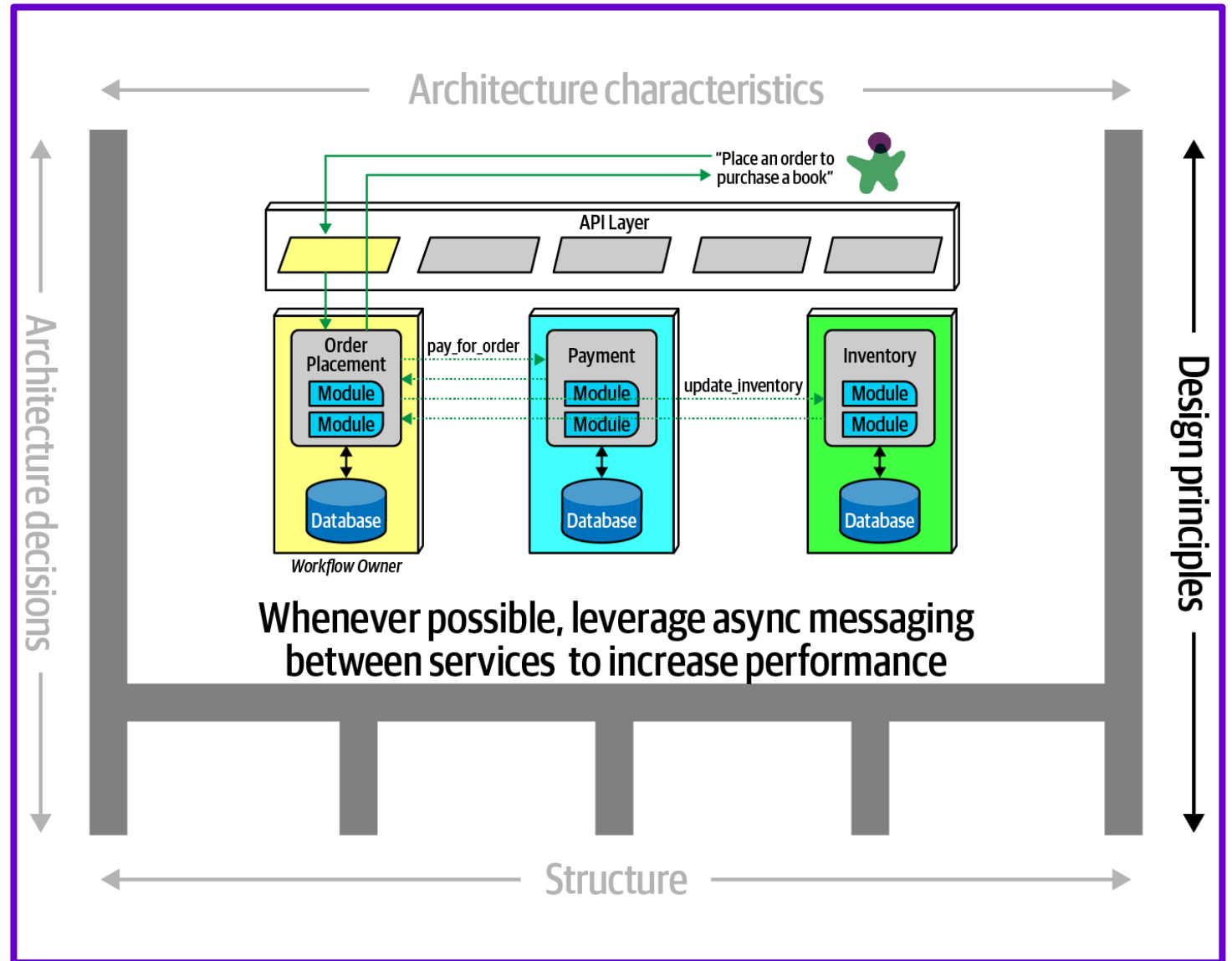
Defining Software Architecture

Architecture decisions are rules for constructing systems



Defining Software Architecture

Design principles are guidelines for constructing systems



Common operational architecture characteristics

Term	Definition
Availability	How long the system will need to be available (if 24/7, steps need to be in place to allow the system to be up and running quickly in case of any failure).
Continuity	Disaster recovery capability.
Performance	Includes stress testing, peak analysis, analysis of the frequency of functions used, capacity required, and response times. Performance acceptance sometimes requires an exercise of its own, taking months to complete.
Recoverability	Business continuity requirements (e.g., in case of a disaster, how quickly is the system required to be on-line again?). This will affect the backup strategy and requirements for duplicated hardware.
Reliability/ safety	Assess if the system needs to be fail-safe, or if it is mission critical in a way that affects lives. If it fails, will it cost the company large sums of money?
Robustness	Ability to handle error and boundary conditions while running if the internet connection goes down or if there's a power outage or hardware failure.
Scalability	Ability for the system to perform and operate as the number of users or requests increases.

Structural architecture characteristics

Term	Definition
Configurability	Ability for the end users to easily change aspects of the software's configuration (through usable interfaces).
Extensibility	How important it is to plug new pieces of functionality in.
Installability	Ease of system installation on all necessary platforms.
Leverageability/ reuse	Ability to leverage common components across multiple products.
Localization	Support for multiple languages on entry/query screens in data fields; on reports, multibyte character requirements and units of measure or currencies.
Maintainability	How easy it is to apply changes and enhance the system?
Portability	Does the system need to run on more than one platform? (For example, does the frontend need to run against Oracle as well as SAP DB?
Supportability	What level of technical support is needed by the application? What level of logging and other facilities are required to debug errors in the system?
Upgradeability	Ability to easily/quickly upgrade from a previous version of this application/solution to a newer version on servers and clients.

1

Introduction to Software Architecture

2

Defining Software Architecture & Architecture Characteristics

3

Layered Architecture Style

4

Microkernel Architecture Style

5

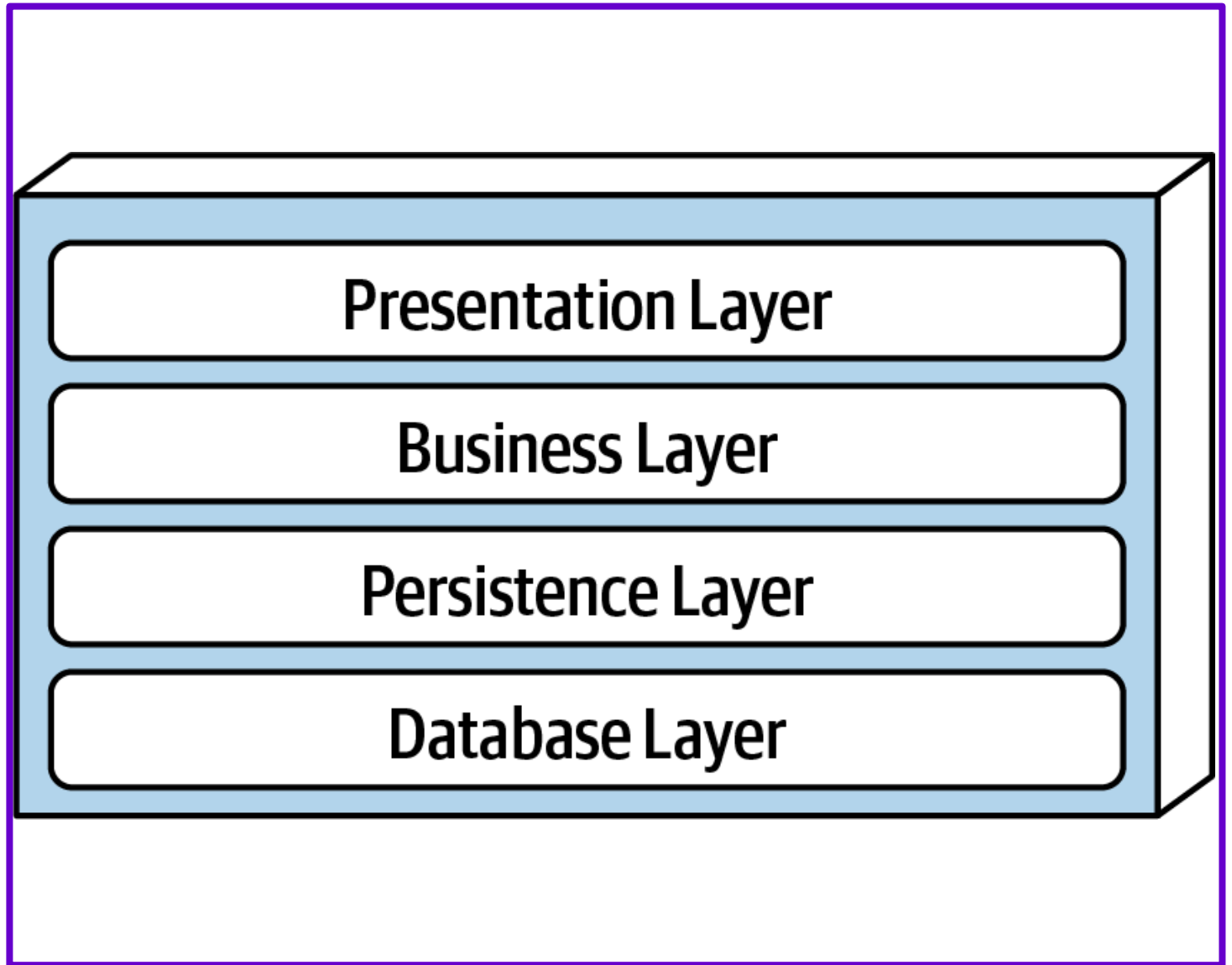
Service-Based Architecture Style

6

Event-Driven Architecture Style

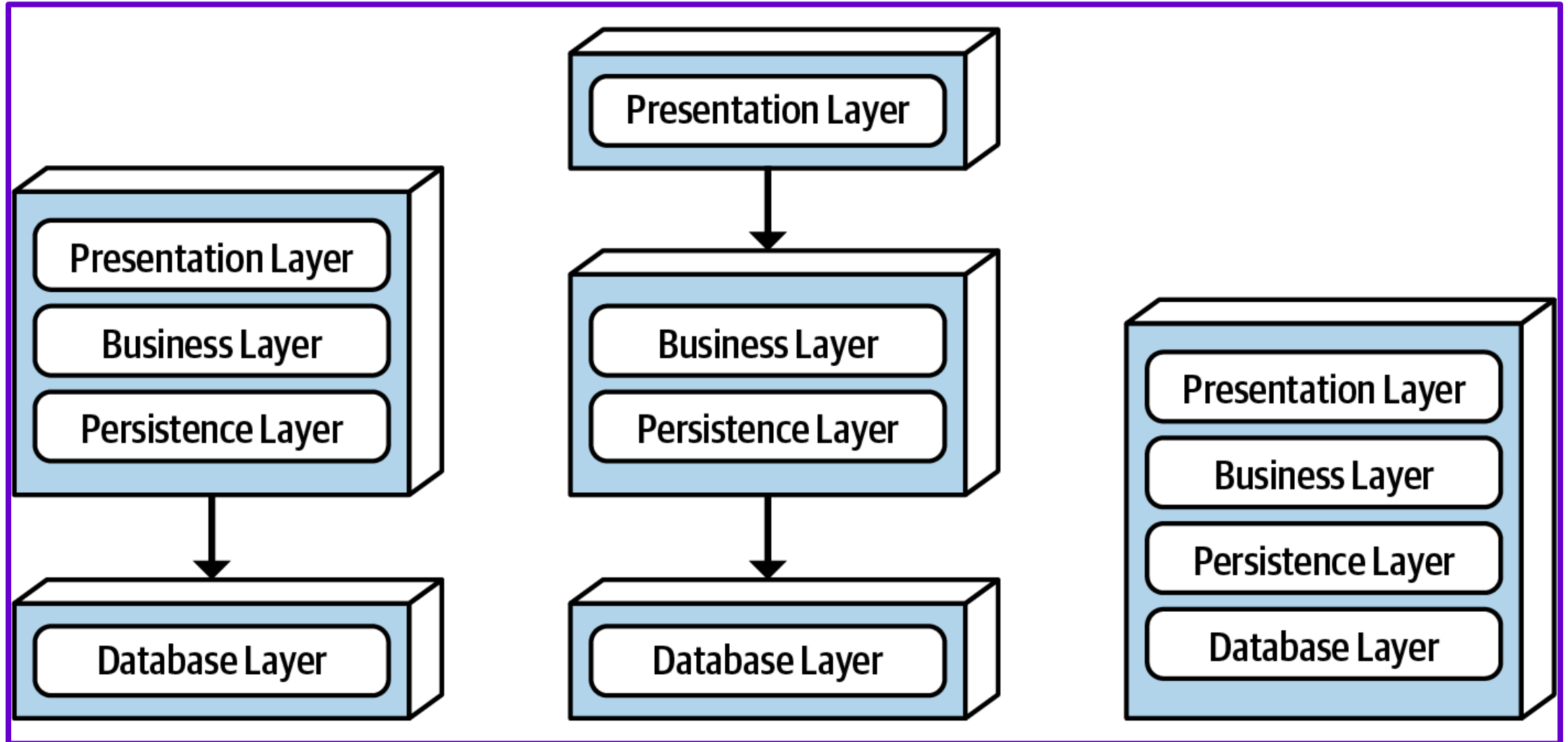
Layered Architecture Style

Standard logical
layers within the
layered architecture
style



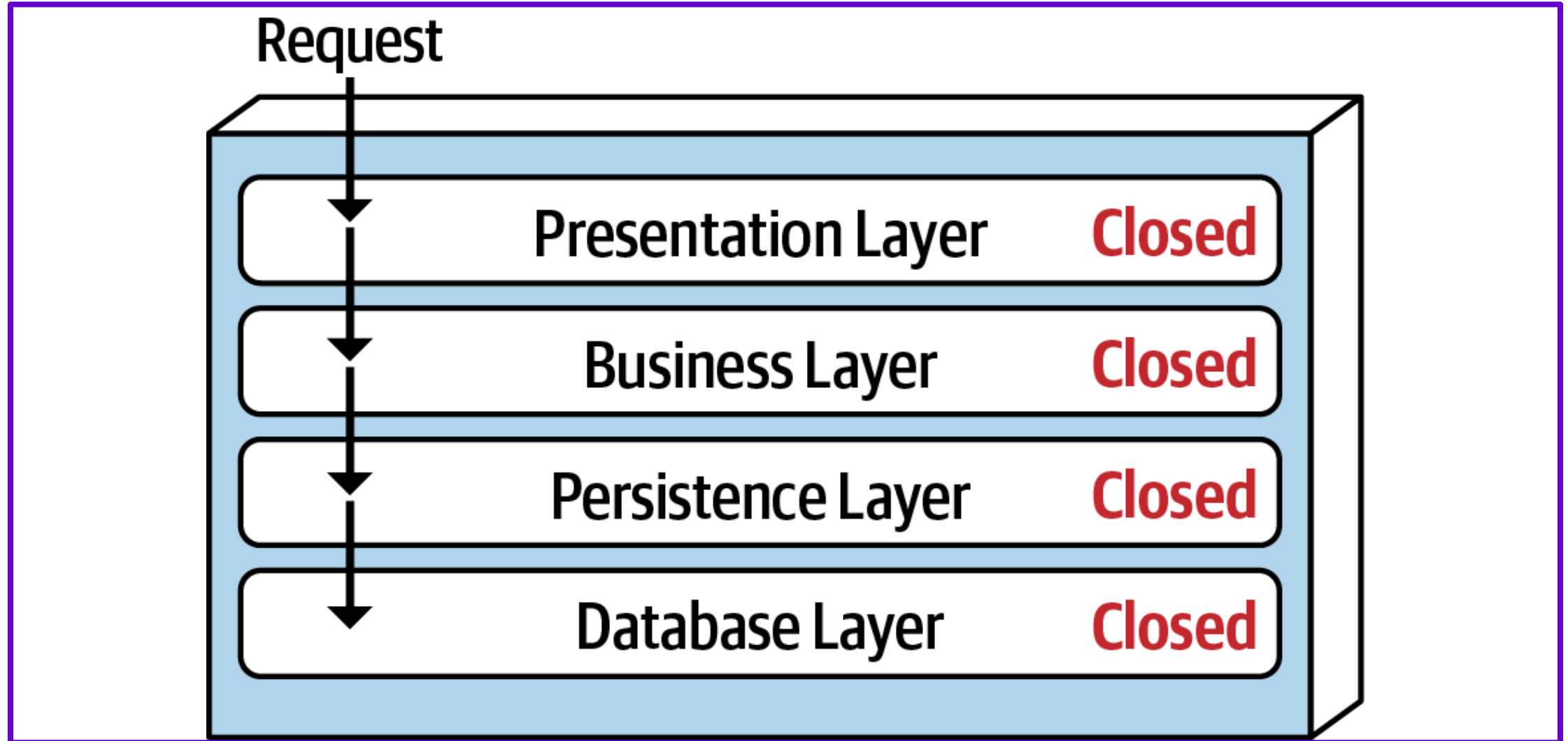
Layered Architecture Style:

Physical topology (deployment) variants



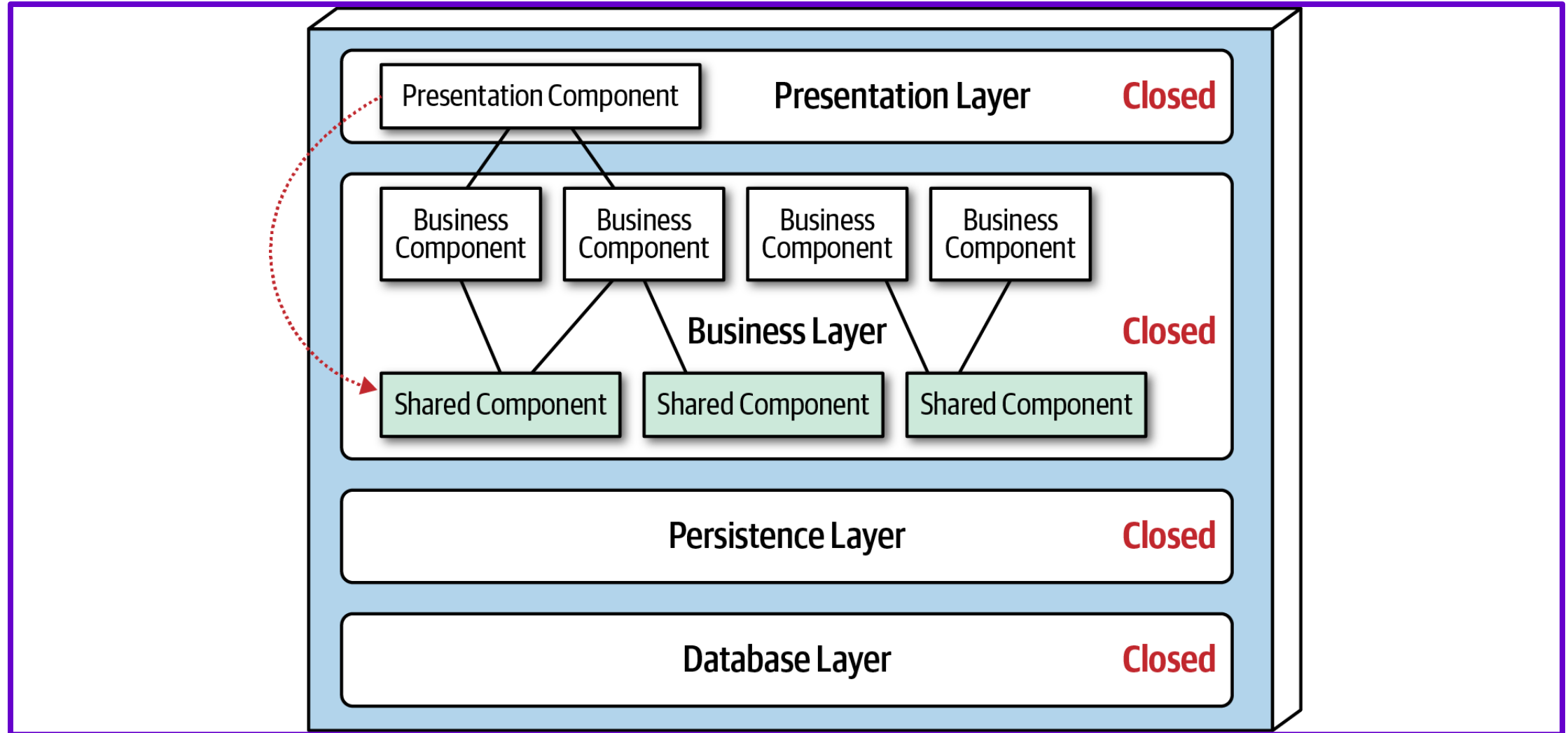
Layered Architecture Style:

Closed layers within the layered architecture



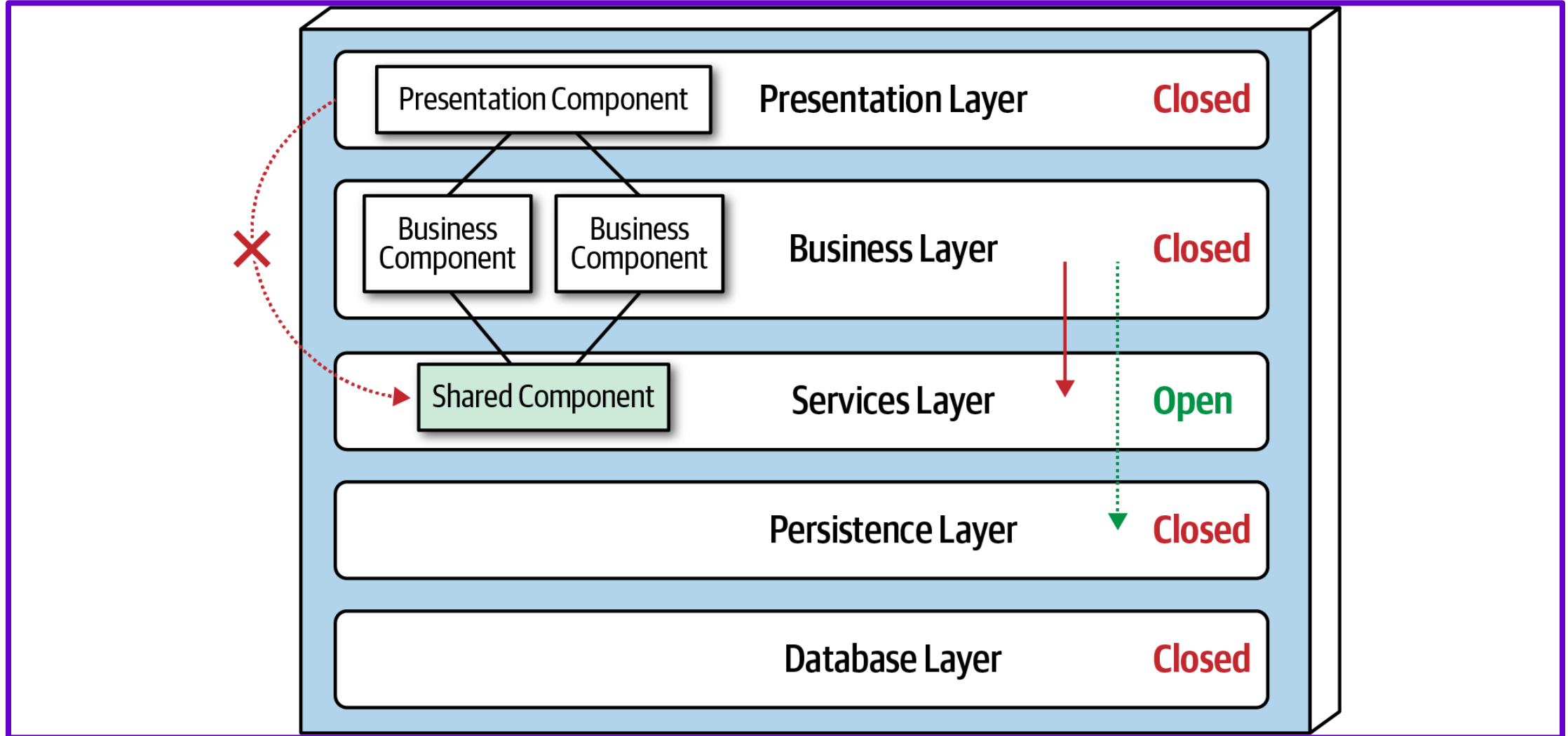
Layered Architecture Style:

Shared objects within the business layer



Layered Architecture Style:

Adding a new services layer to the architecture



1

Introduction to Software Architecture

2

Defining Software Architecture & Architecture Characteristics

3

Layered Architecture Style

4

Microkernel Architecture Style

5

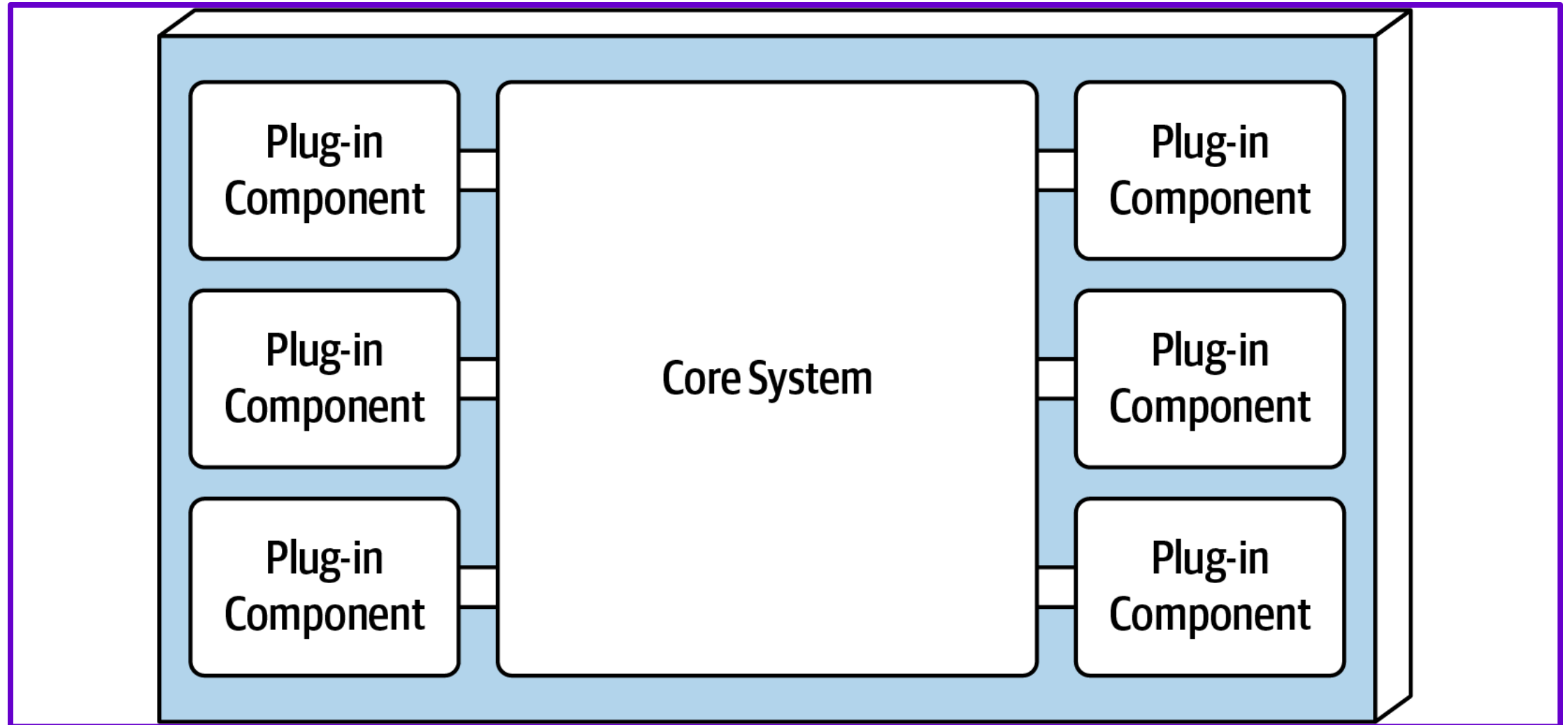
Service-Based Architecture Style

6

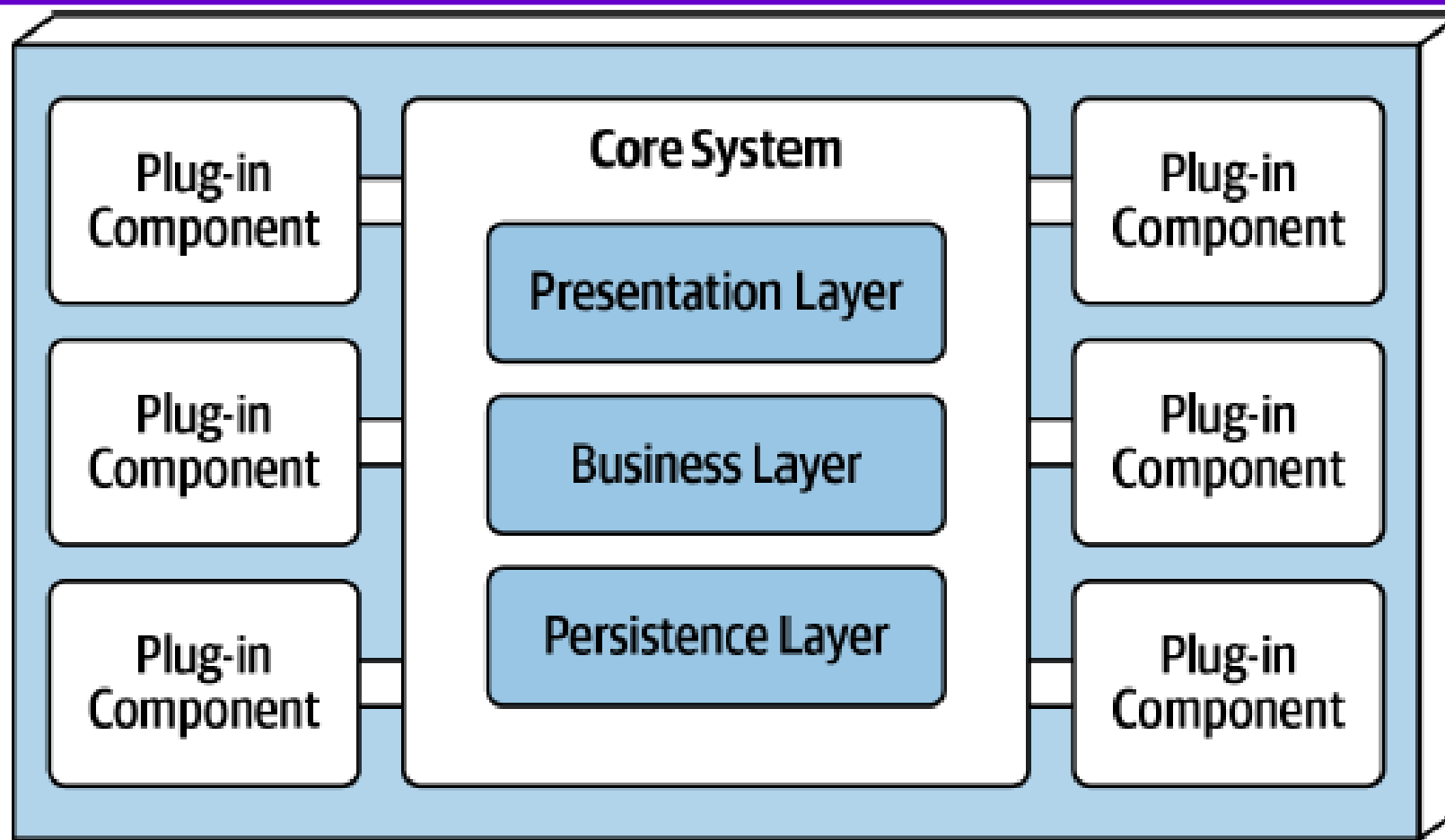
Event-Driven Architecture Style

Microkernel architecture :

Basic components of the style

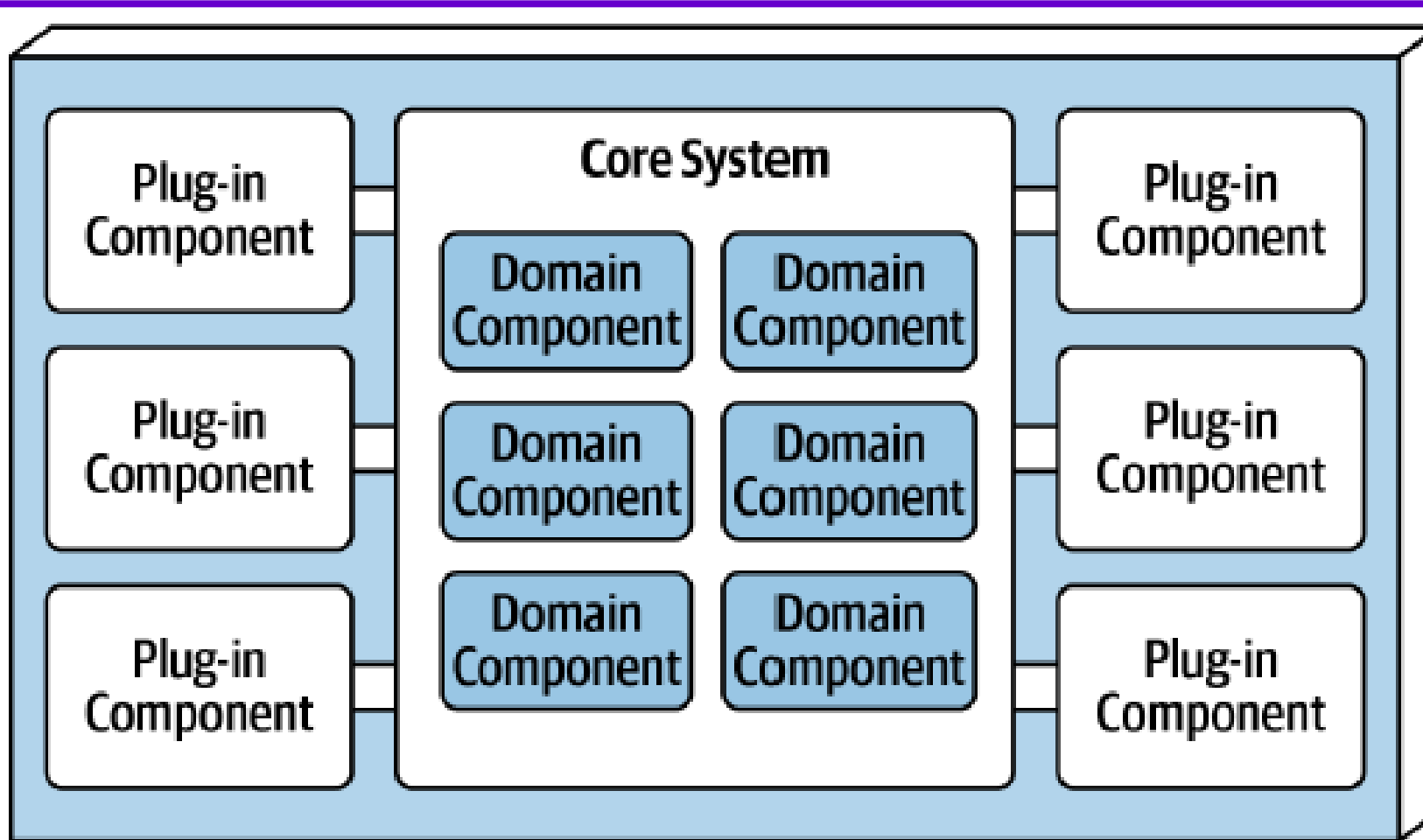


Variations of the microkernel architecture core system



Layered Core System (Technically Partioned)

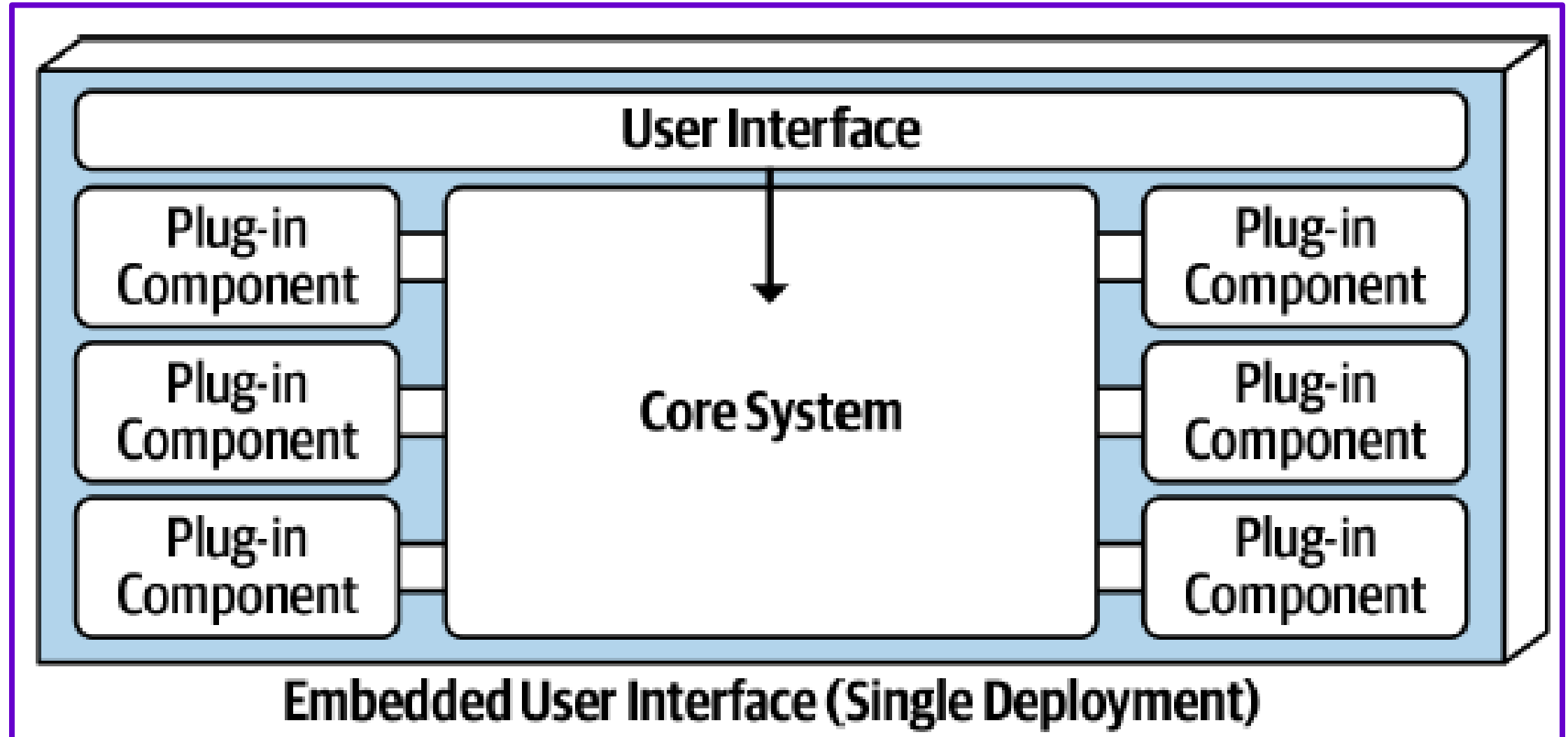
Variations of the microkernel architecture core system



Modular Core System (Domain Partitioned)

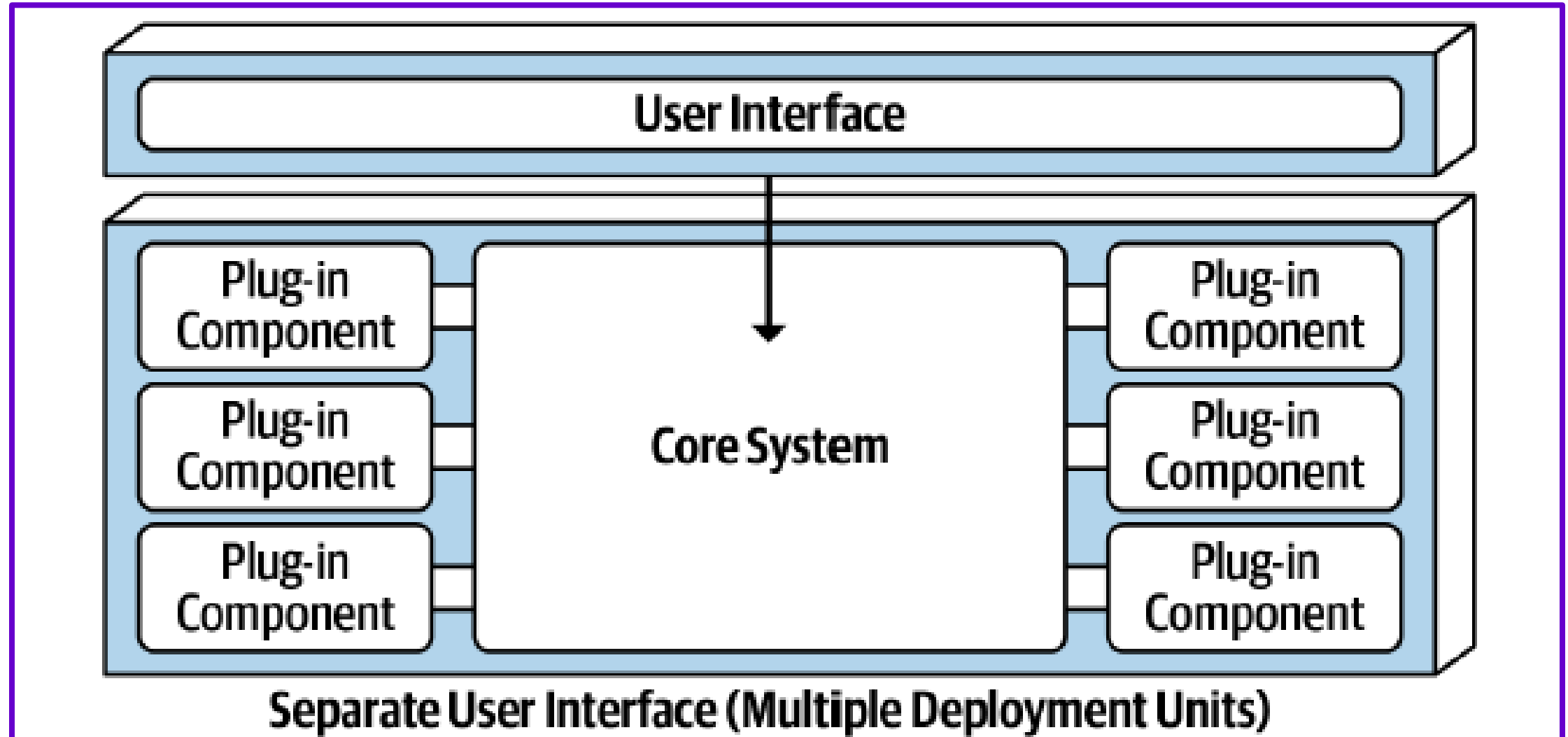
Microkernel architecture :

User interface variants



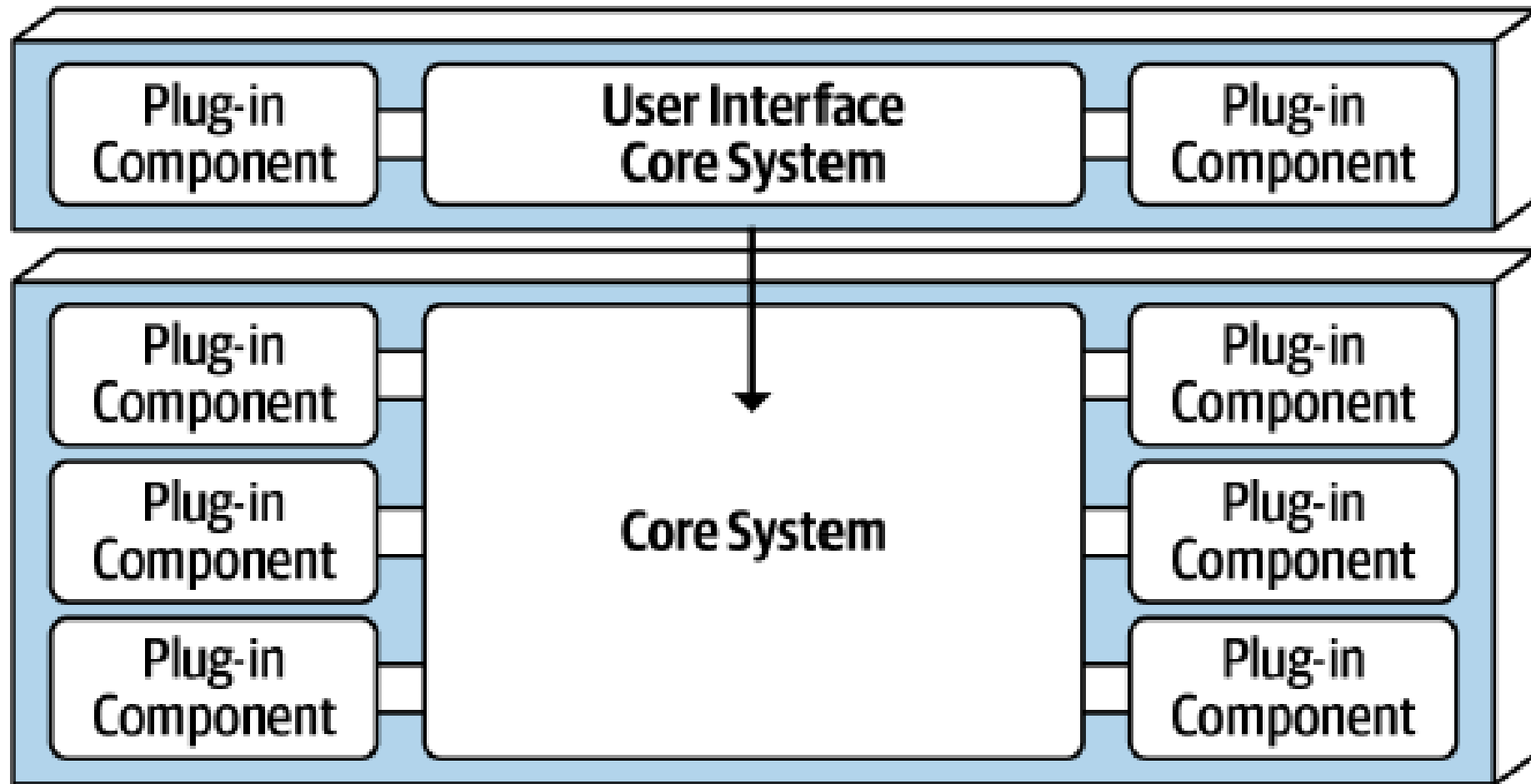
Microkernel architecture :

Basic components of the style



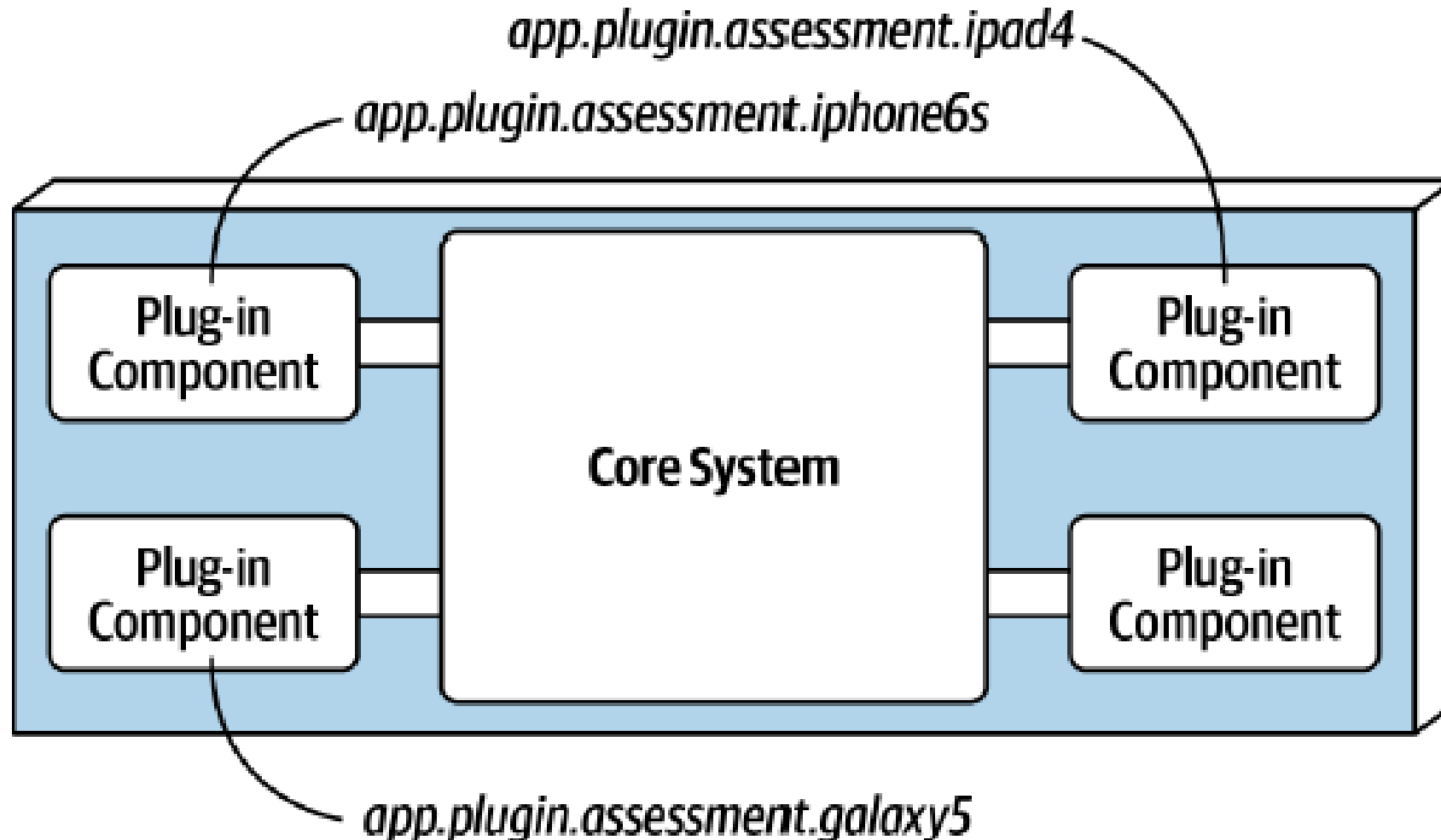
Microkernel architecture :

Basic components of the style



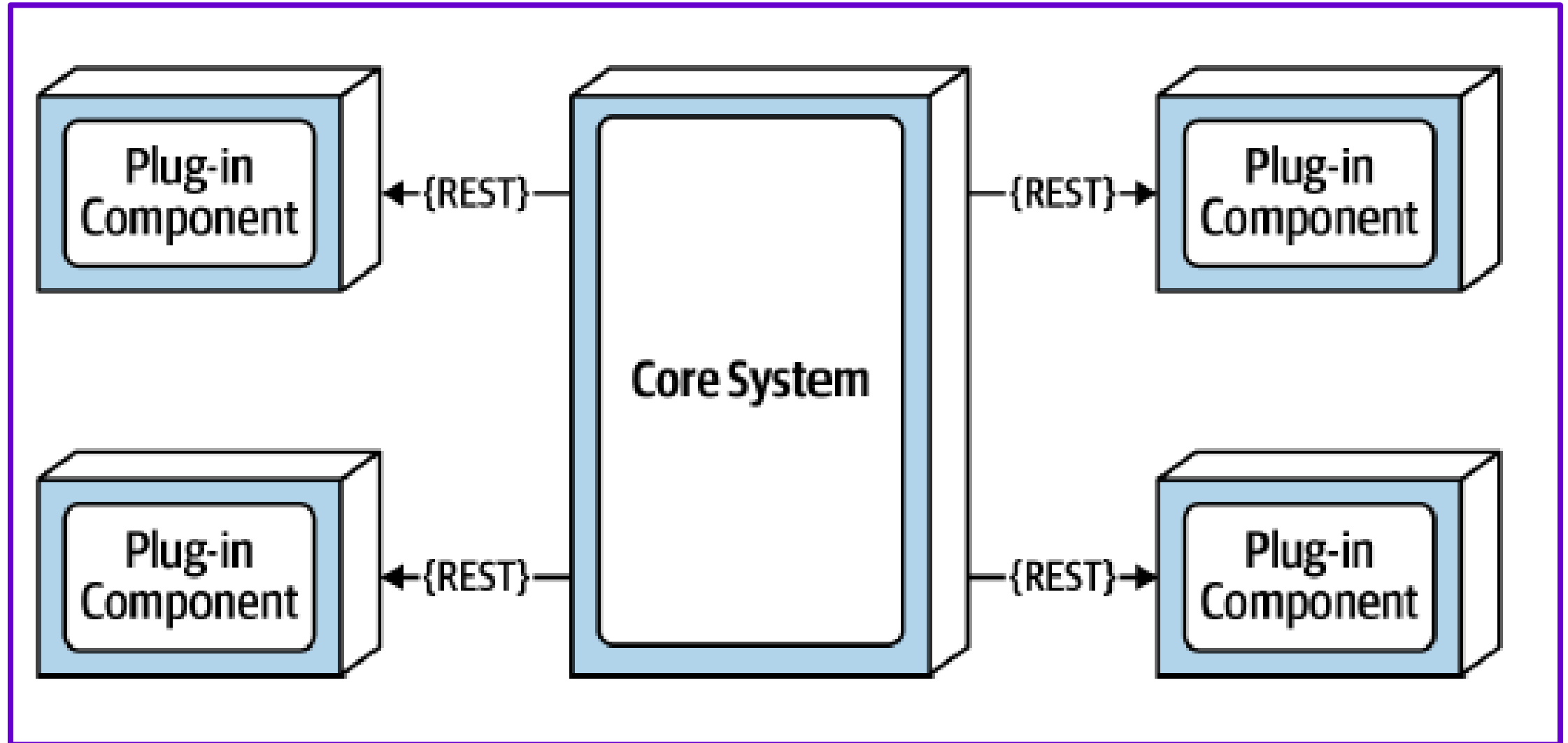
Separate User Interface (Multiple Deployment Units, Both Microkernel)

Microkernel architecture : Plug-In Components



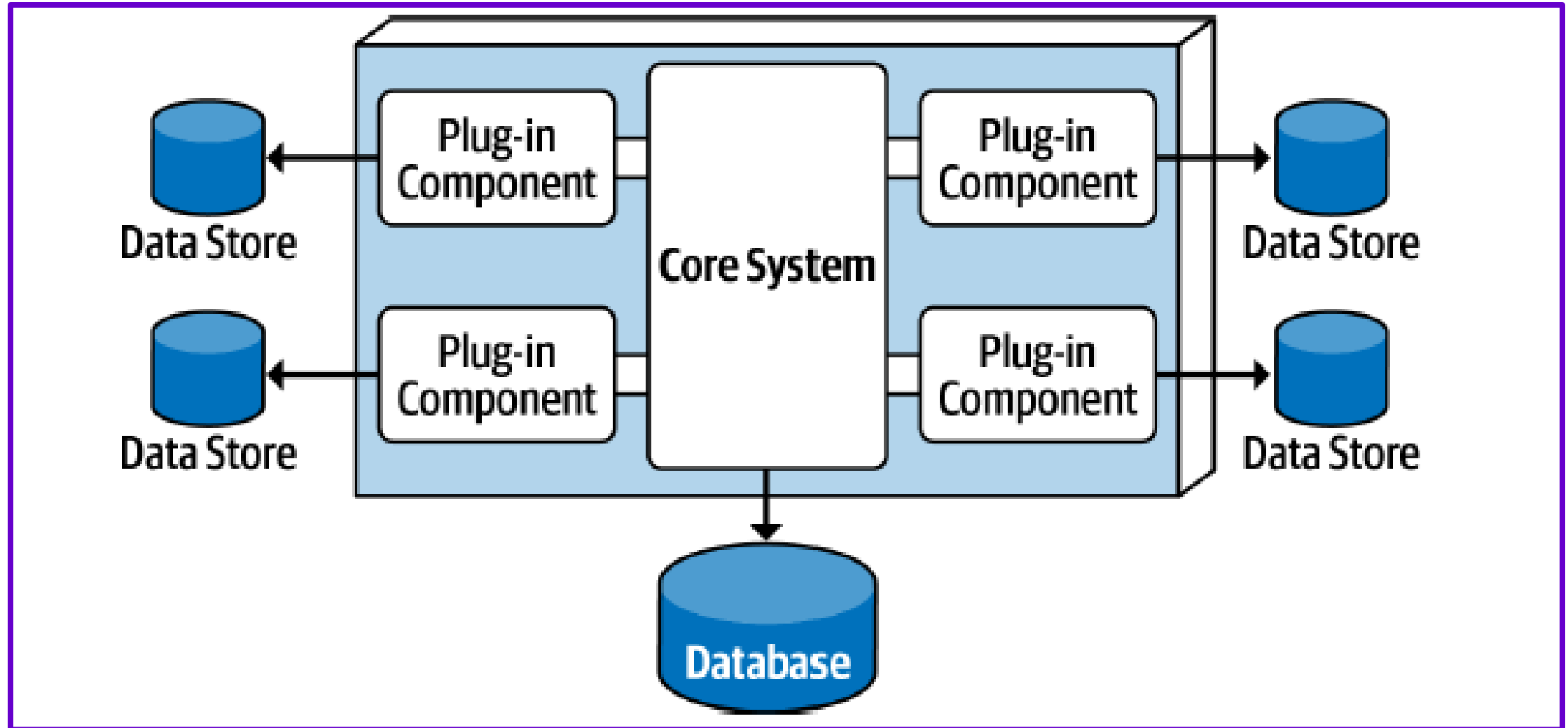
Microkernel architecture :

Remote plug-in access using REST



Microkernel architecture :

Plug-in components can own their own data store



1

Introduction to Software Architecture

2

Defining Software Architecture & Architecture Characteristics

3

Layered Architecture Style

4

Microkernel Architecture Style

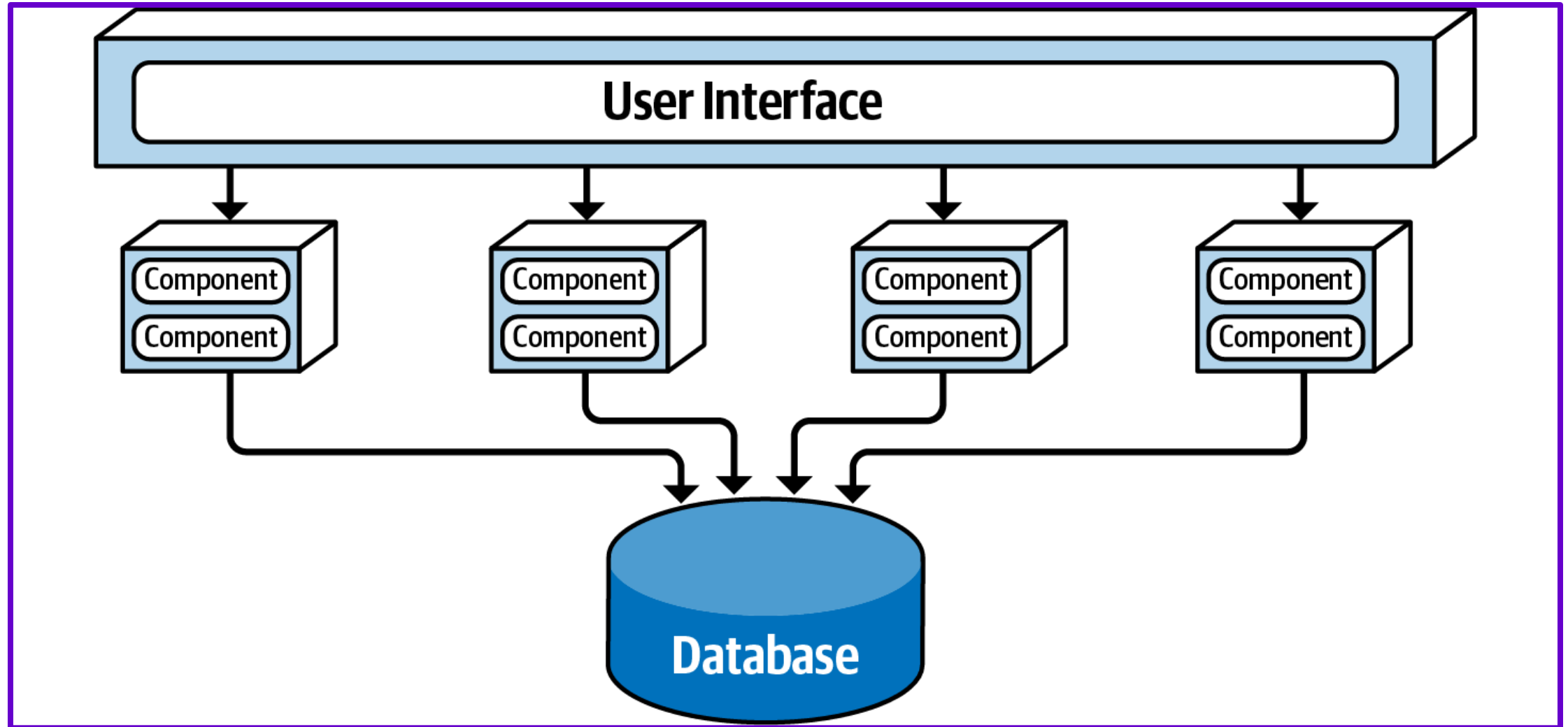
5

Service-Based Architecture Style

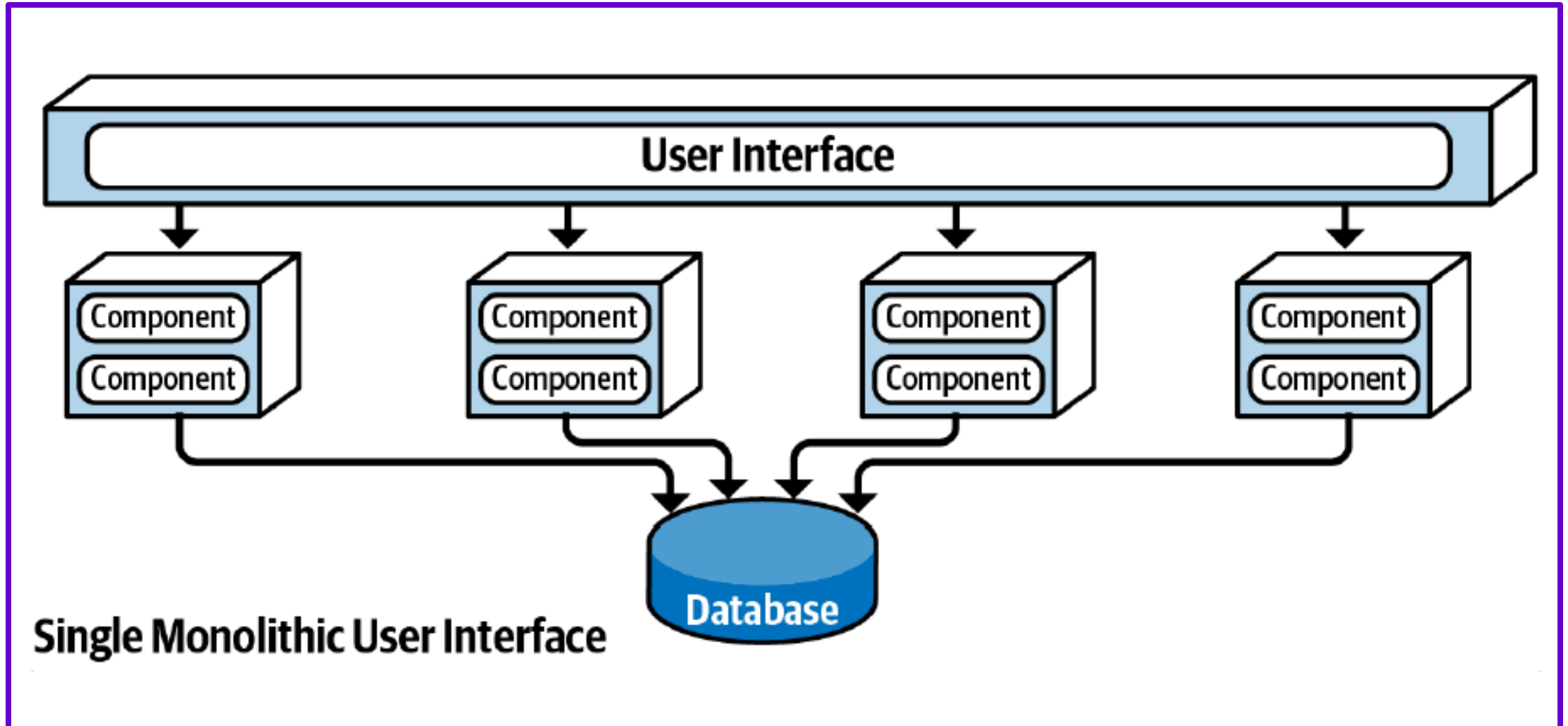
6

Event-Driven Architecture Style

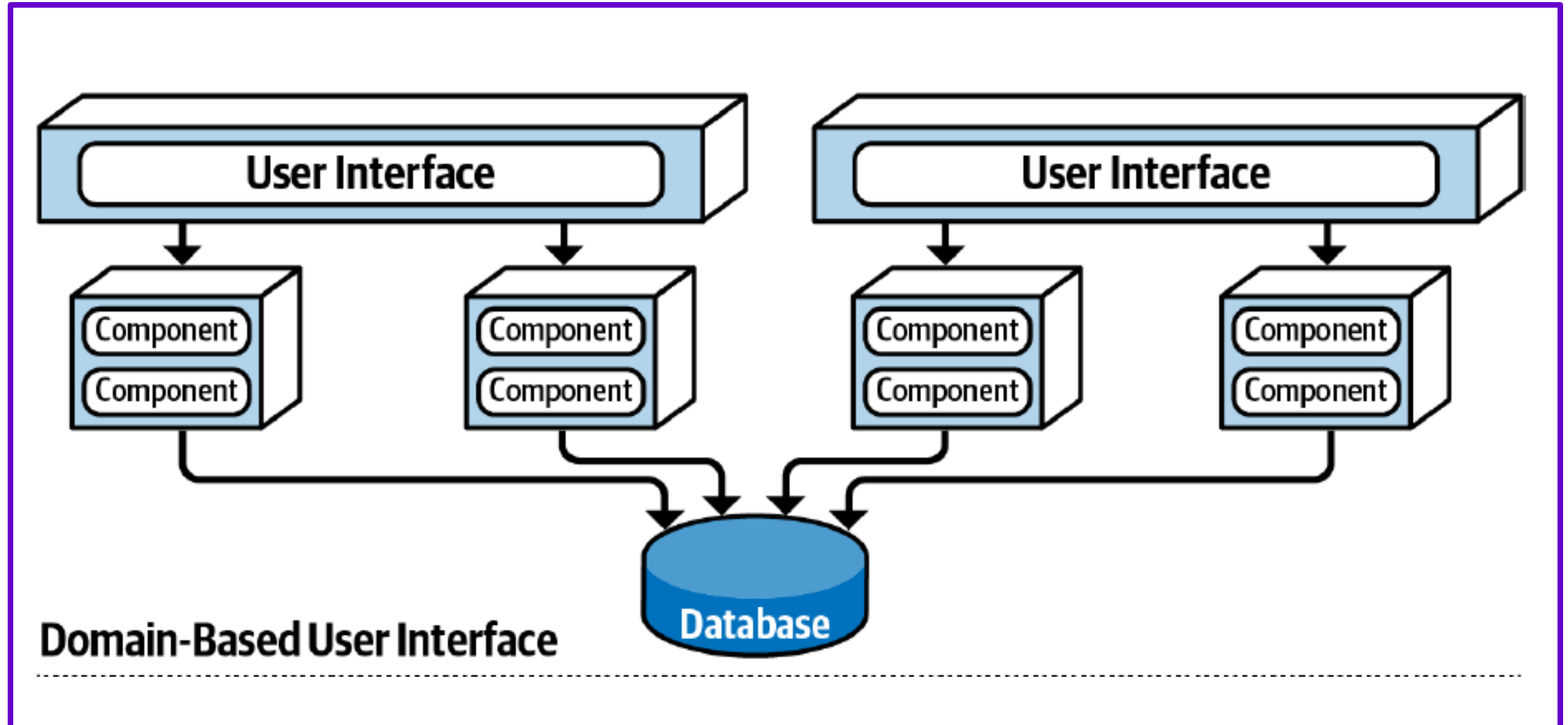
Service Oriented Architecture



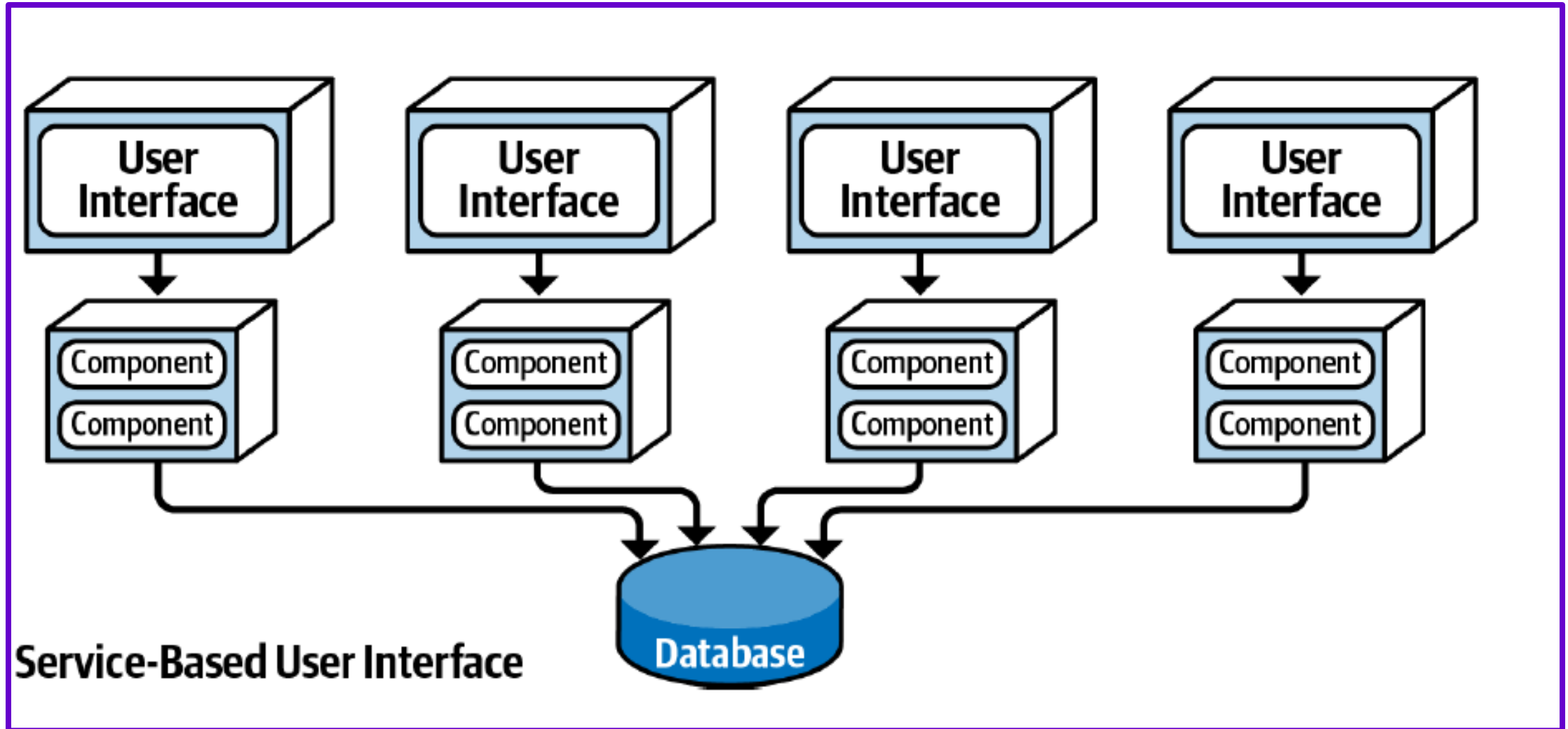
Service Oriented Architecture: User Interface Variants 1



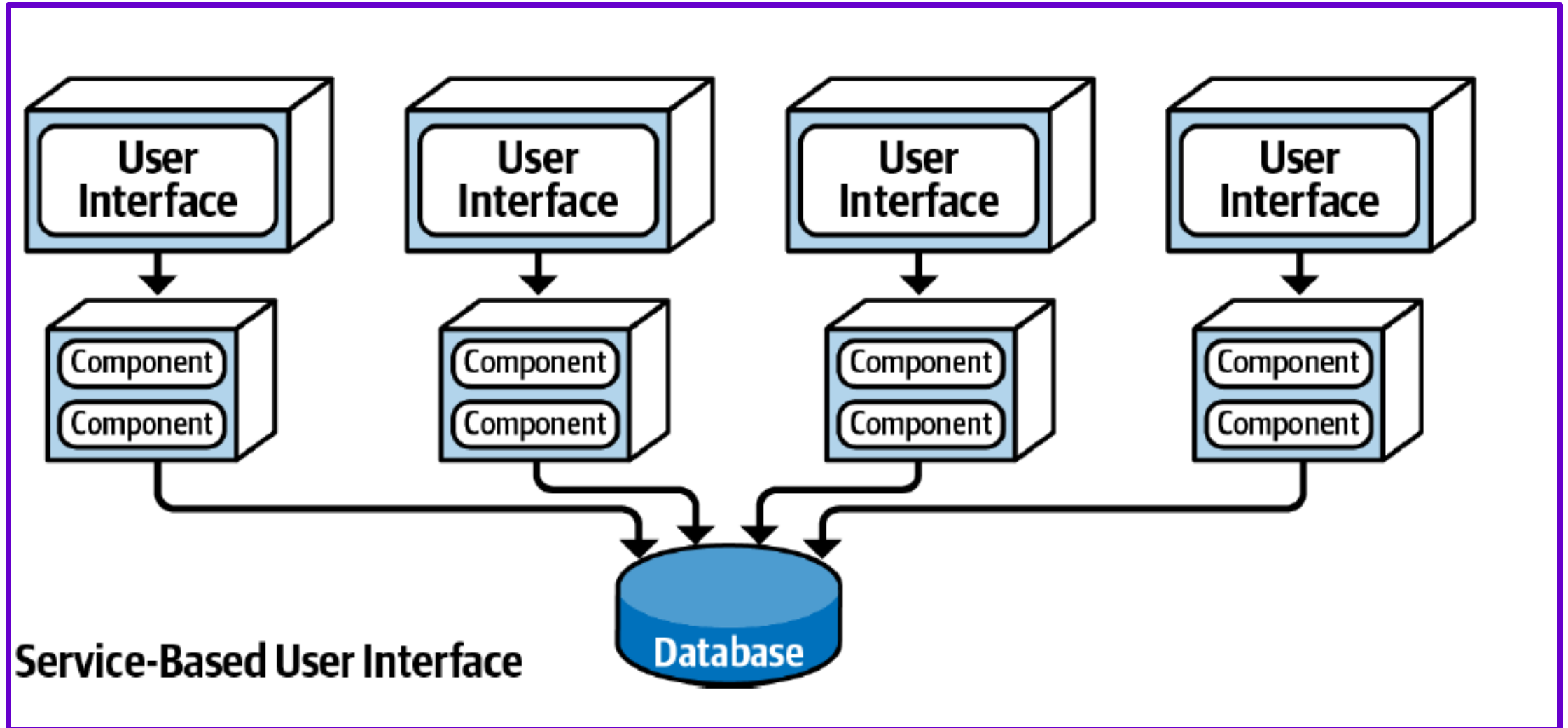
Service Oriented Architecture: User Interface Variants 2



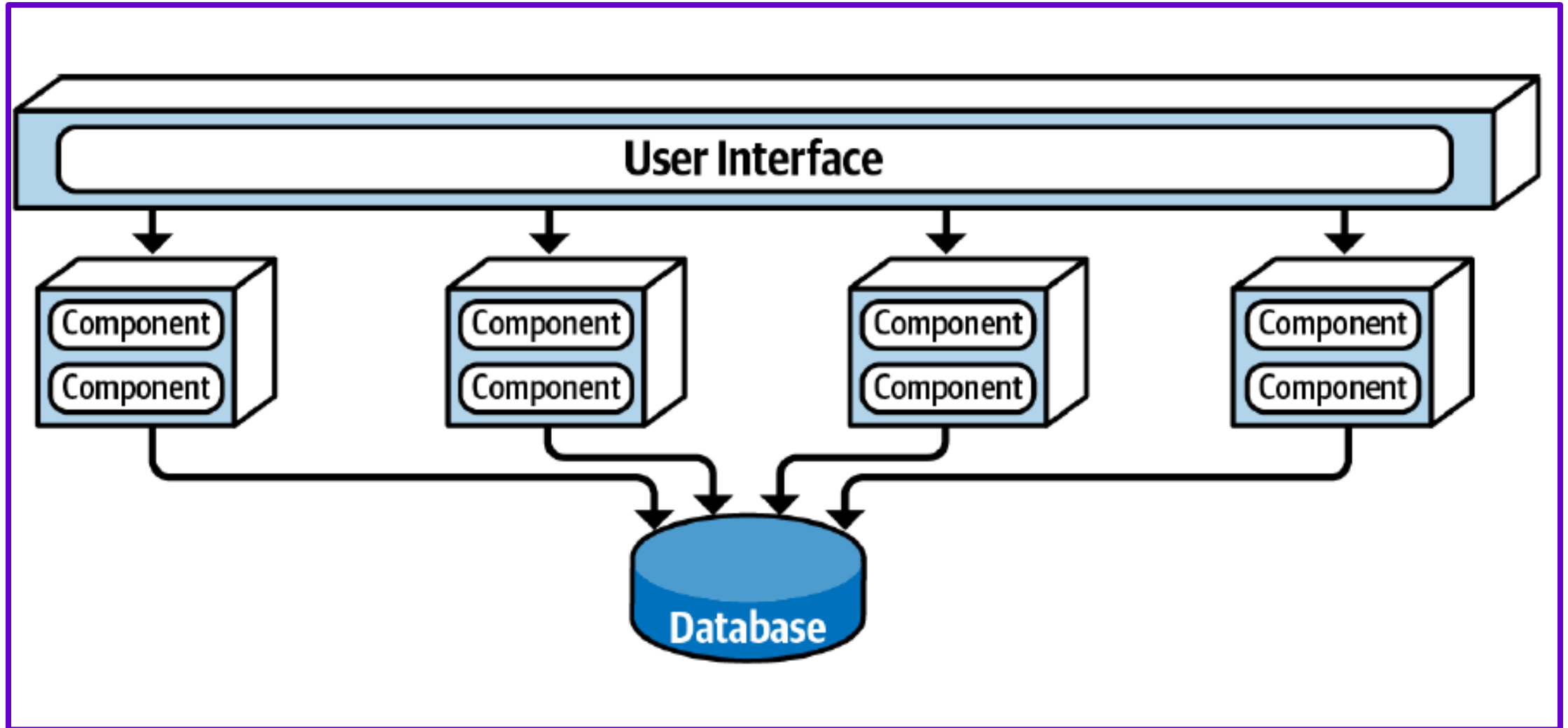
Service Oriented Architecture: User Interface Variants 3



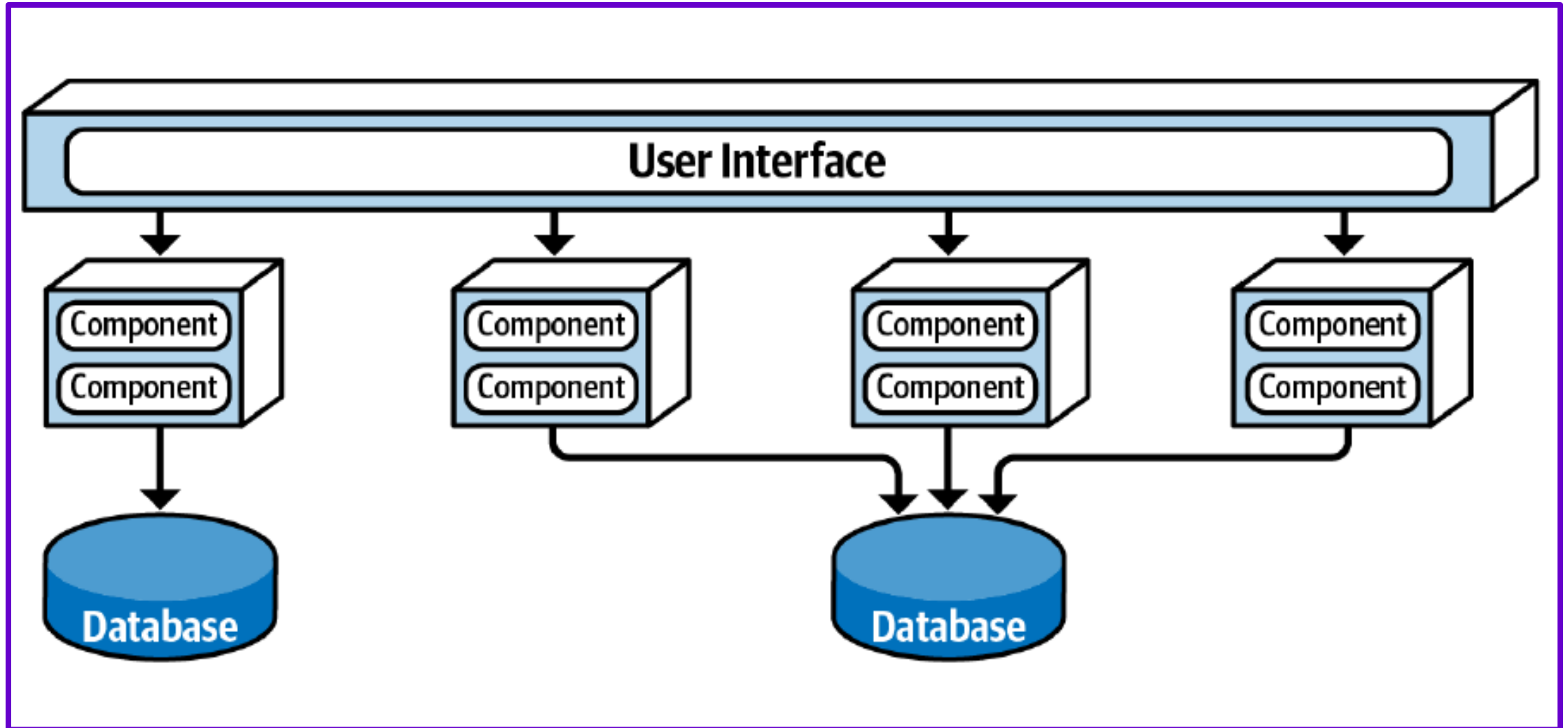
Service Oriented Architecture: User Interface Variants 3



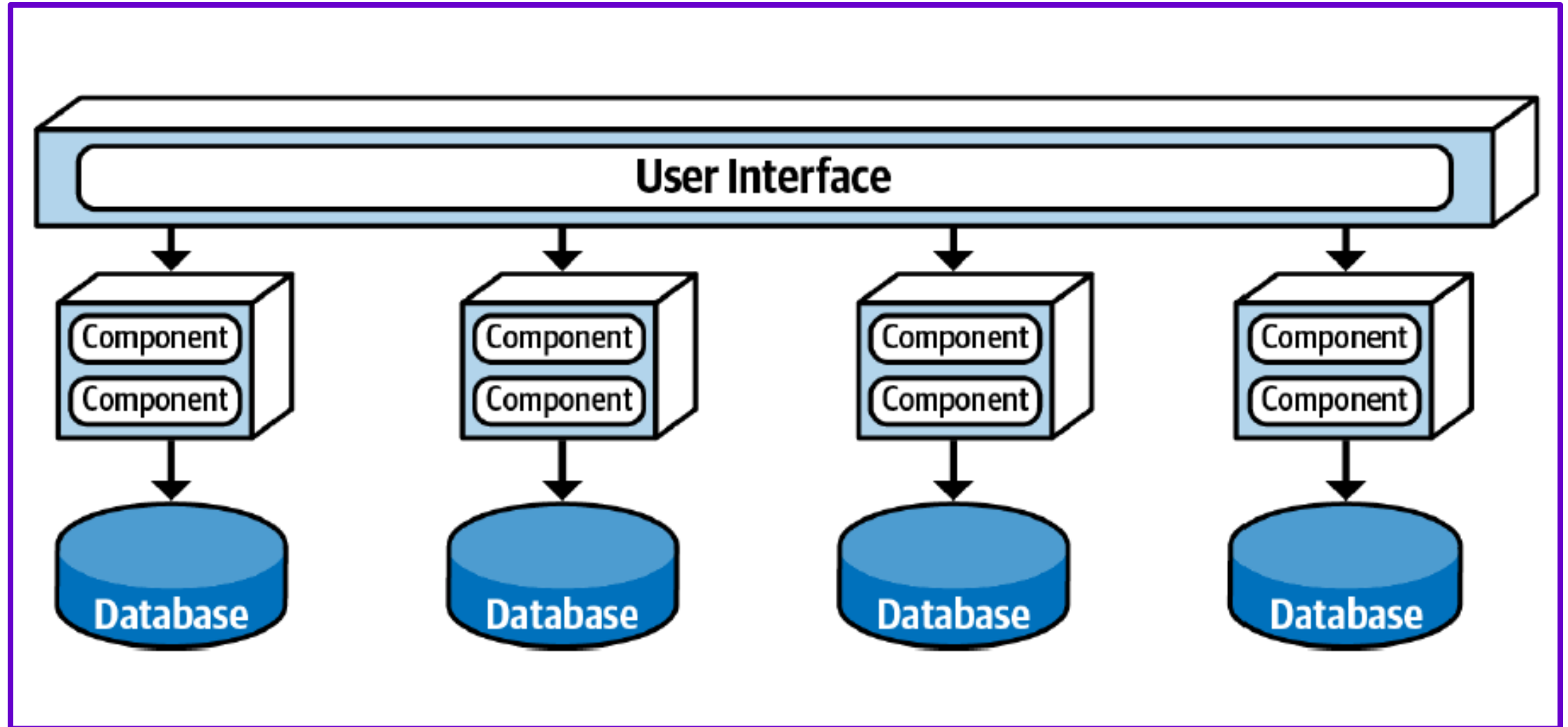
Service Oriented Architecture: Database Variants 1



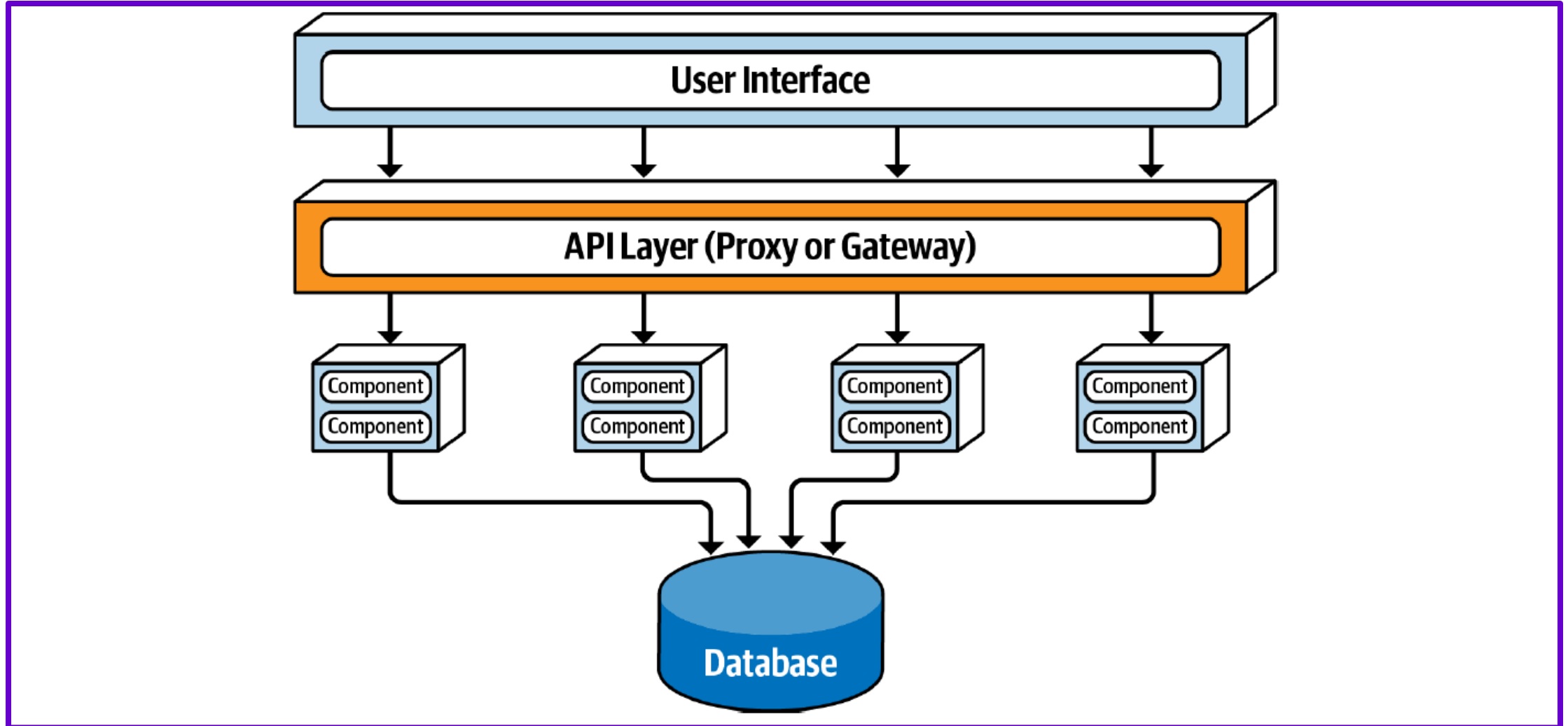
Service Oriented Architecture: Database Variants 2



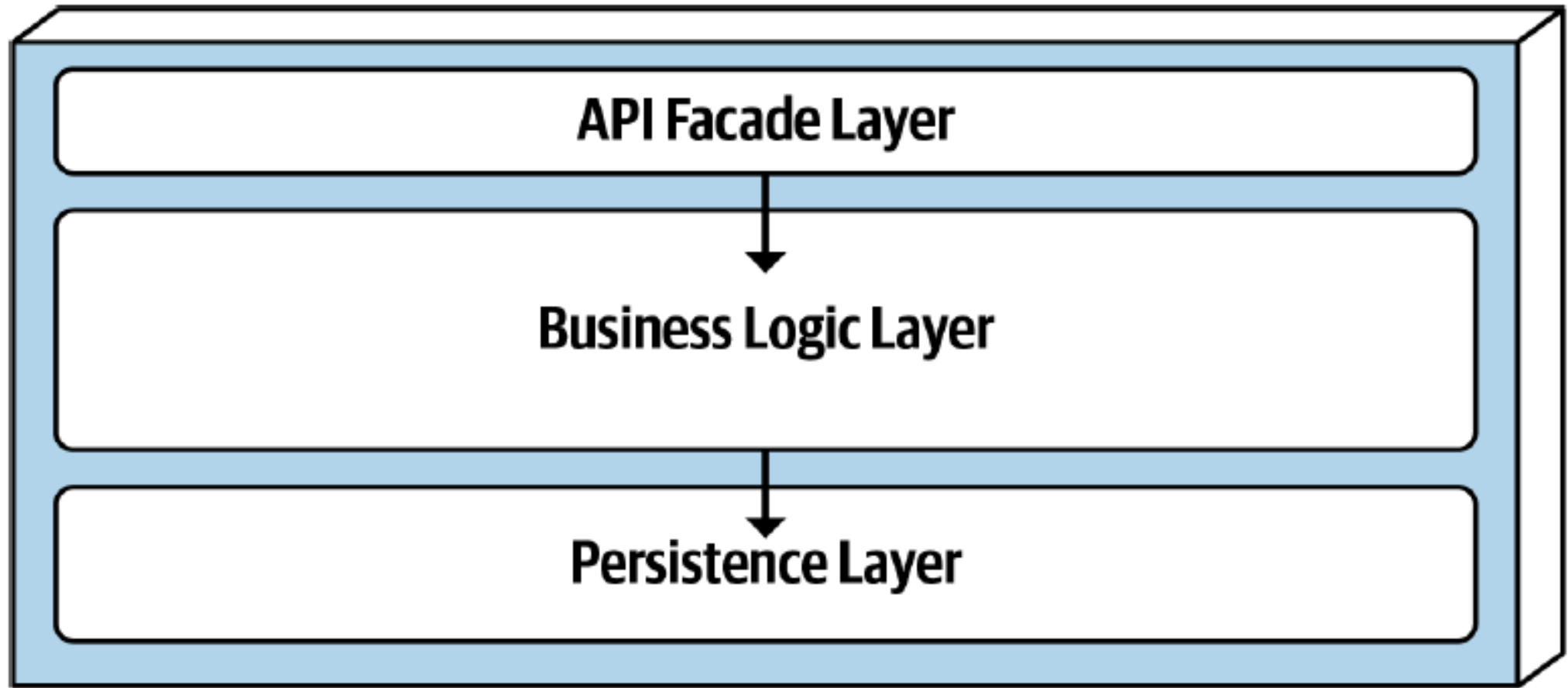
Service Oriented Architecture: Database Variants 3



Adding an API layer between the user interface and domain services



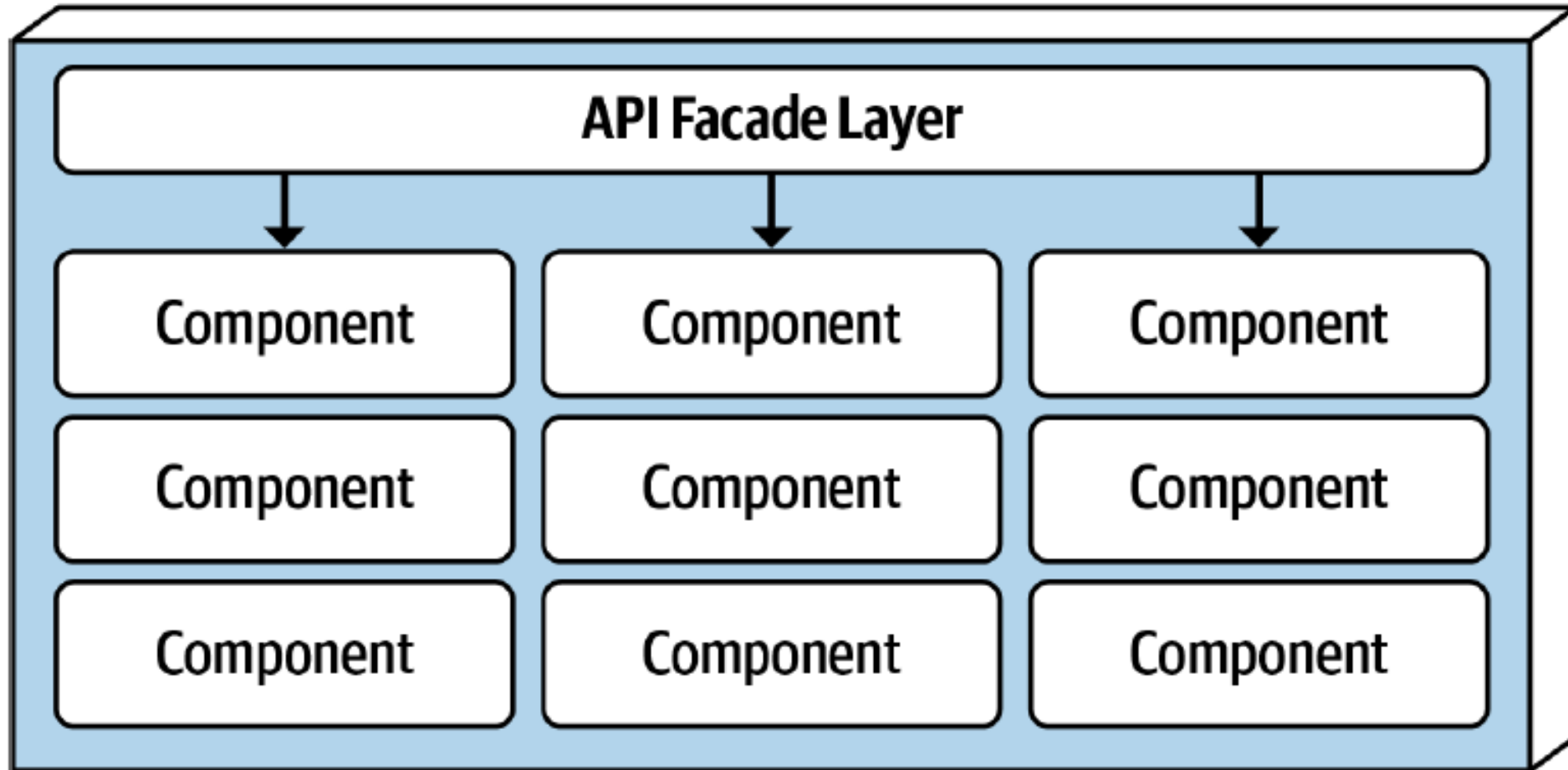
Domain service design variants



Layered design (technical partitioning)

Domain service design variants

Domain design (domain partitioning)



1

Introduction to Software Architecture

2

Defining Software Architecture & Architecture Characteristics

3

Layered Architecture Style

4

Microkernel Architecture Style

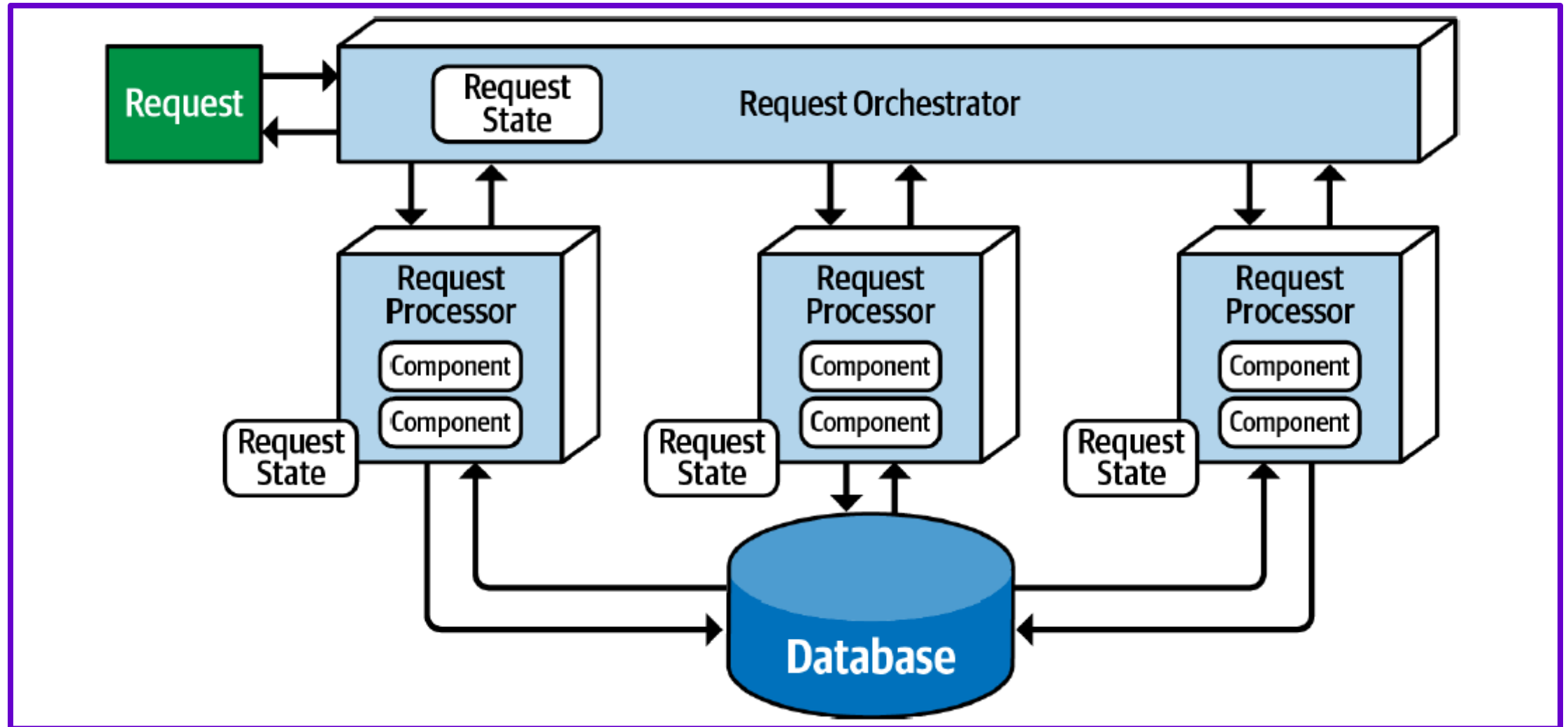
5

Service-Based Architecture Style

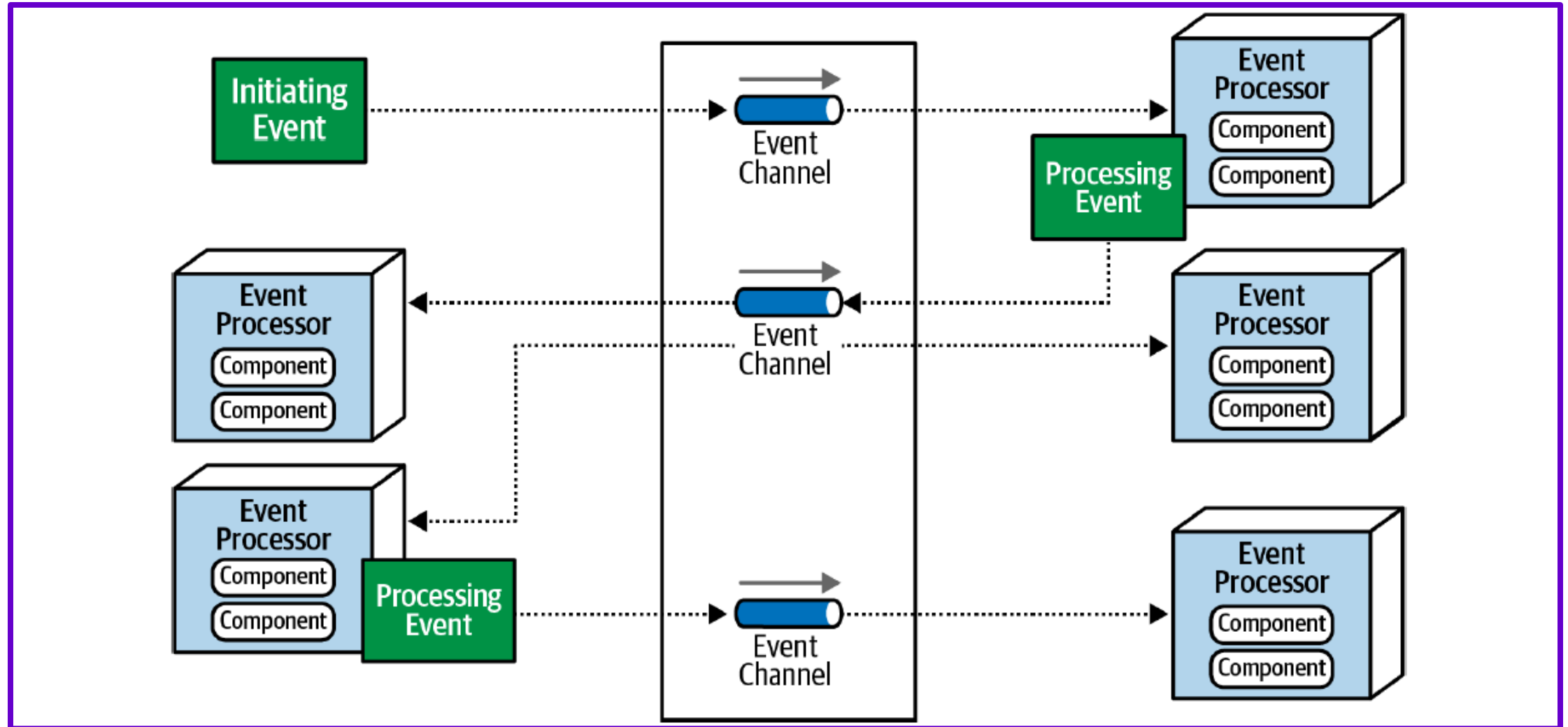
6

Event-Driven Architecture Style

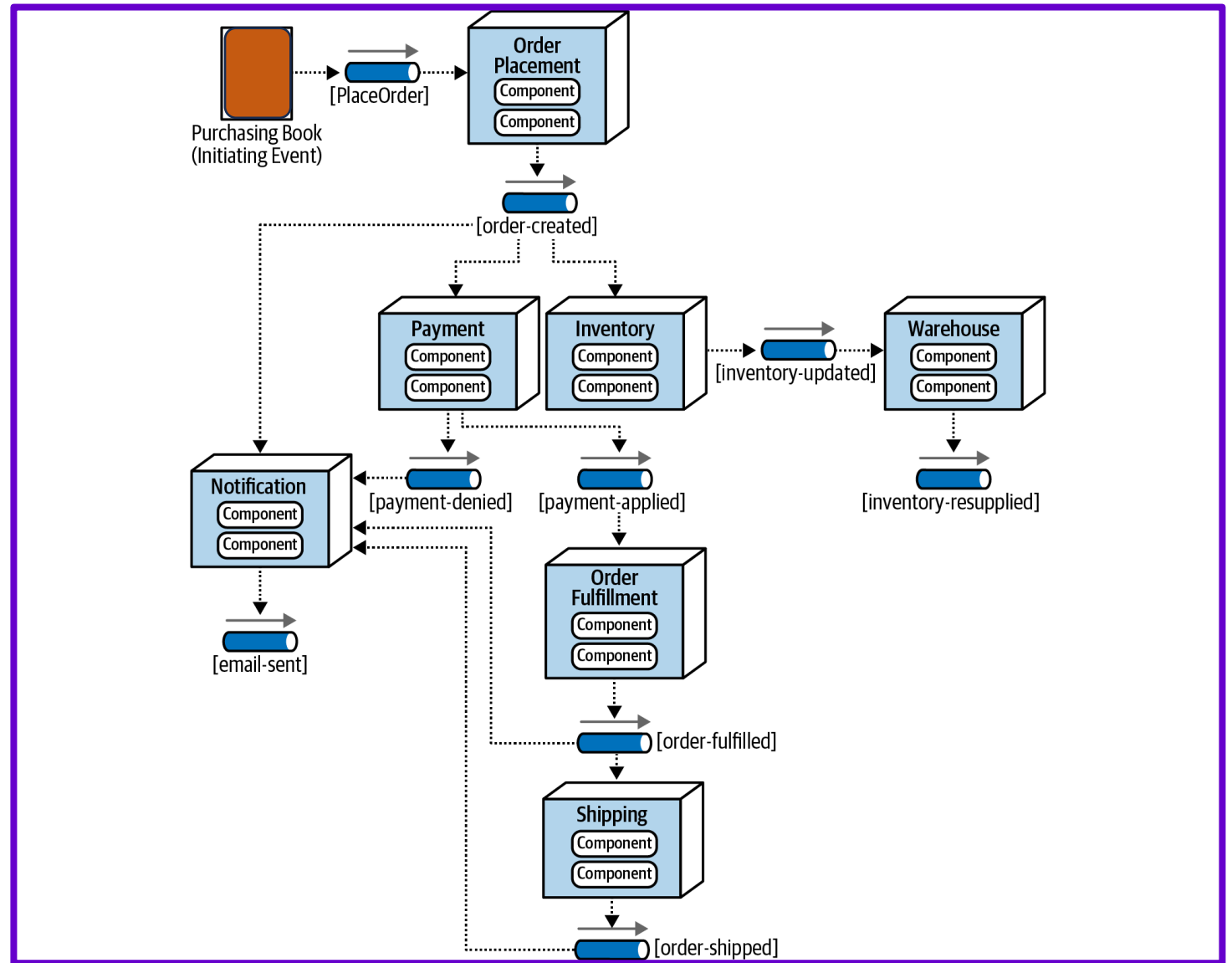
Request-based model



Broker topology



Example of the broker topology



Summary

Layered architecture characteristics ratings

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1
Deployability	★
Elasticity	★
Evolutionary	★
Fault tolerance	★
Modularity	★
Overall cost	★★★★★
Performance	★★
Reliability	★★★
Scalability	★
Simplicity	★★★★★
Testability	★★

Summary

Microkernel architecture characteristics ratings

Architecture characteristic	Star rating
Partitioning type	Domain and technical
Number of quanta	1
Deployability	★ ★ ★
Elasticity	★
Evolutionary	★ ★ ★
Fault tolerance	★
Modularity	★ ★ ★
Overall cost	★ ★ ★ ★ ★
Performance	★ ★ ★
Reliability	★ ★ ★
Scalability	★
Simplicity	★ ★ ★ ★
Testability	★ ★ ★

Summary

Service-based architecture characteristics ratings

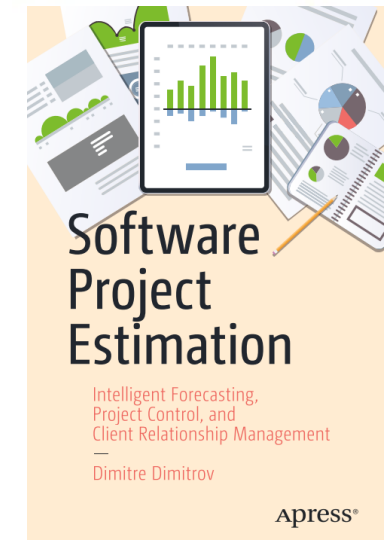
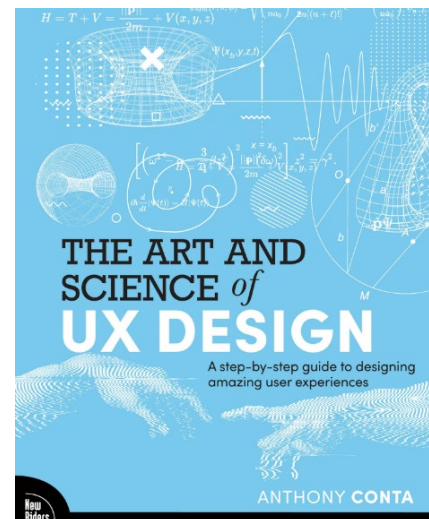
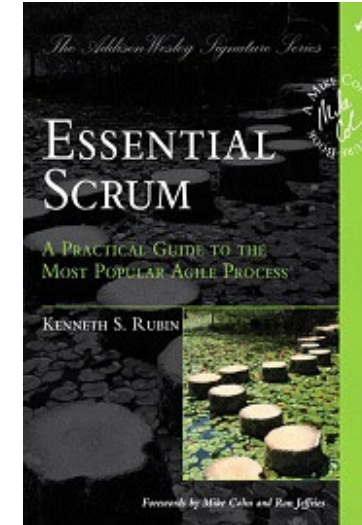
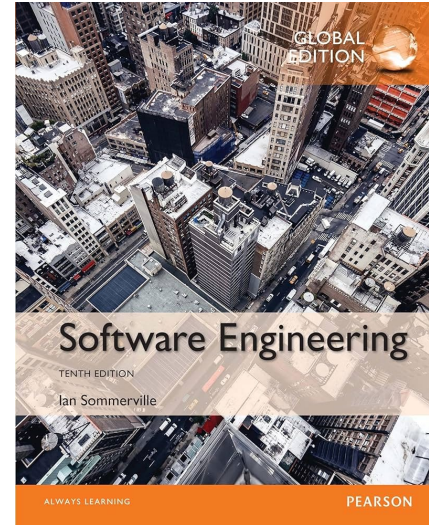
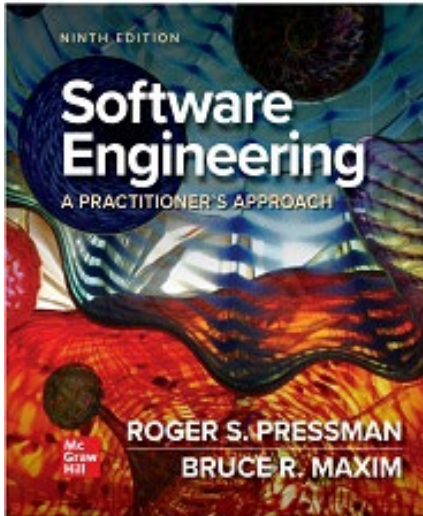
Architecture characteristic	Star rating
Partitioning type	Domain
Number of quanta	1 to many
Deployability	★★★★
Elasticity	★★
Evolutionary	★★★
Fault tolerance	★★★★
Modularity	★★★★
Overall cost	★★★★
Performance	★★★
Reliability	★★★★
Scalability	★★★
Simplicity	★★★
Testability	★★★★

Summary

Event-driven architecture characteristics ratings

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1 to many
Deployability	★ ★ ★
Elasticity	★ ★ ★
Evolutionary	★ ★ ★ ★ ★
Fault tolerance	★ ★ ★ ★ ★
Modularity	★ ★ ★ ★
Overall cost	★ ★ ★
Performance	★ ★ ★ ★ ★
Reliability	★ ★ ★
Scalability	★ ★ ★ ★ ★
Simplicity	★
Testability	★ ★

Course References



Course References

- ***Designing User Interfaces***, Michal Malewicz & Diana Malewice, 2020
- ***UI Design Styles: Trends and Design Patterns***, Michal Malewicz & Diana Malewice, 2020
- ***What UX Is Really About :Introducing a Mindset for Great Experiences***, Celia Hodent, CRC Press, 2022
- ***Lean UX: Designing Great Products with Agile Teams 3rd Edition***, Jeff Gothelf & Josh Seiden, O'Reilly, 2021
- ***Laws of UX: Using Psychology to Design Better Products & Services***, Jon Yablonski, O'Reilly, 2020
- ***Designing and Prototyping Interfaces with Figma***, Fabio Staiano, Packet Publishing, 2022

Course References

- [1] Green, M. D. (2016). Scrum: Novice to Ninja: Methods for Agile, Powerful Development, SitePoint.
- [2] Ockerman, S. and S. Reindl (2019). Mastering professional scrum: A practitioner's guide to overcoming challenges and maximizing the benefits of agility, Addison-Wesley Professional.
- [3] Martin, R. C. (2019). Clean Agile, Pearson Education.
- [4] Hall, G. M. (2017). Adaptive Code: Agile coding with design patterns and SOLID principles, Microsoft
- [5] سامانی‌پور، علی. (۲۰۱۸). آموزش اسکرام برای مدیریت چابک فرایند توسعه اپلیکیشن های وب و موبایل،
فرادرس